

Multimode Rabi dynamics: finite models, experiments and patch analysis

Documentation of the `rabi-truncation` repository

28 September 2026

Abstract

The repository evolves pure states with one shared engine, enumerated photon occupation bases and sparse Hamiltonians. This document specifies the finite models actually solved, derives their preparation and measurement formulas, explains the common-space embedding used for fidelities, and documents the Python and notebook interfaces. The comparison with the student implementation separates unitary changes of convention, changes of the finite physical model, and corrections to numerical comparisons. Numbered source listings are followed by comments tied to their current lines. The common `full` space used for overlaps is implicit; choosing a separately propagated comparison trajectory does not require propagating or allocating that ambient space.

Contents

1	Finite model, state preparation and calculated quantities	1
1.1	Periodic grid, parameter controls and photon spaces	1
1.2	Preparation, projected Hamiltonian and evolution	4
1.3	Observables, exact fidelities and physical interpretation	7
2	Notebook control, resources and source-line commentary	10
2.1	Shared configuration and callable Python experiments	10
2.2	Correspondence between controls and resource costs	13
2.3	Numbered code and comments on individual operations	15
3	Comparison with the student implementation and validation	50
3.1	Exact convention conversions and finite-model changes	50
3.2	Mathematical verification and limits of the conclusions	53

1 Finite model, state preparation and calculated quantities

1.1 Periodic grid, parameter controls and photon spaces

Notation and units. Throughout, $\hbar = c = 1$. Length and time have inverse-energy units. The TLS energy origin is its ground state: its free Hamiltonian is $\omega_0 |e\rangle \langle e|$, rather than $\omega_0 \sigma_z/2$. These differ by a scalar, which matters when reporting an absolute energy. The symbols used below are:

Symbol	Meaning in the implementation
L	Periodic box length, supplied in <code>param_atom</code> .
k_m, ω_m	Signed momentum and nonnegative frequency $\omega_m = k_m $.
M	Number of retained modes; after selection this means the selected count, unless explicitly described as the base-grid count.
N, B, d	<code>n_max</code> , number of photon configurations, and joint ket dimension $d = 2B$.
$\mathbf{n}, \nu$	Occupation tuple and total photon number $\nu = \sum_m n_m$.
D, f_m, u_m	Physical coupling, chosen profile and complex annihilation coefficient, defined in equation (30).
k_0, σ_k, x_0	Packet momentum centre, width convention and initial Fourier position parameter.
x_{tls}	Parameter in the imposed annihilation phase; its plotted interaction coordinate is derived in equation (47).
T, dt, N_t	Final time, nominal output spacing and number of output times, including both endpoints.

Grid and the two controls. For a periodic box of length $L > 0$,

$$\varphi_j(x) = \frac{e^{ik_j x}}{\sqrt{L}}, \quad \Delta k = \frac{2\pi}{L}, \quad k_j = j\Delta k, \quad j \in \mathbb{Z}, \quad \omega_j = |k_j|. \quad (1)$$

Indeed, $\varphi_j(x + L) = \varphi_j(x)$ and $\int_{x_*}^{x_*+L} \varphi_j^* \varphi_\ell dx = \delta_{j\ell}$. Signed momenta enter spatial phases; their absolute values enter the dispersion and square-root profile. Modes are sorted by signed momentum. There is no automatic deletion of the zero mode.

When `CTRL_M_EXPLICIT=True`, the inputs are L and a positive odd integer M . Setting $J = (M - 1)/2$ gives

$$\mathcal{K}_M = \{j\Delta k : -J \leq j \leq J\}, \quad |\mathcal{K}_M| = M, \quad k_{\text{IR},\text{eff}} = 0, \quad k_{\text{UV},\text{eff}} = \frac{M-1}{2} \Delta k. \quad (2)$$

The cutoff dictionary is inactive. An even M raises **ValueError**; it is never rounded, decremented or made asymmetric. $M = 1$ gives $\mathcal{K} = \{0\}$, not a resonant single-mode cavity at ω_0 . Booleans and floating-point values, including 5.0, are rejected as mode-count inputs; Python and NumPy integer types are accepted.

When `CTRL_M_EXPLICIT=False`, the inputs are L and finite radial cutoffs $0 \leq k_{\text{IR}} \leq k_{\text{UV}}$:

$$\mathcal{K}_{\text{IR},\text{UV}} = \{j\Delta k : k_{\text{IR}} \leq |j|\Delta k \leq k_{\text{UV}}\}. \quad (3)$$

For strictly positive indices, in exact arithmetic,

$$j_{\min} = \max\left(1, \left\lceil \frac{k_{\text{IR}}}{\Delta k} \right\rceil\right), \quad j_{\max} = \left\lfloor \frac{k_{\text{UV}}}{\Delta k} \right\rfloor, \quad (4)$$

$$M_{\text{eff}} = 2 \max(0, j_{\max} - j_{\min} + 1) + \mathbf{1}_{k_{\text{IR}}=0}. \quad (5)$$

The code subtracts 10^{-12} before the ceiling and adds 10^{-12} before the floor, protecting numerically boundary-aligned points. This tolerance is on the dimensionless cutoff-to-spacing ratios. Zero is included if and only if the input IR equals zero exactly. A positive IR excludes it through the band definition even when that IR is very small. An empty band is an error. The resulting count can be even when zero is excluded; the odd- M condition belongs only to the explicit-count branch.

Thus specifying L and M implicitly fixes a UV cutoff, while specifying L and cutoffs implicitly fixes the count. These descriptions coincide for a symmetric contiguous grid containing zero, but not for arbitrary selected subsets or positive-IR bands. Continuous cutoffs are not uniquely recoverable from a mode set: with IR zero, the same $\mathcal{K}_M$ is obtained whenever

$$J\Delta k \leq k_{\text{UV}} < (J+1)\Delta k \quad (6)$$

in exact arithmetic. The estimator reports effective radial extrema, without claiming that the requested cutoff originally equalled an extremum. It reports the smallest nonzero momentum separately: effective IR zero does not mean zero spacing between the nonzero modes.

Additional mode selection. Selection is applied after the base grid is resolved. With $w_p = \text{photon_window}$ and $w_a = \text{atom_window}$ it retains the union

$$|k - k_0| \leq w_p \sigma_k \quad \text{or} \quad |k - \omega_0| \leq w_a \sigma_k \quad \text{or} \quad |k + \omega_0| \leq w_a \sigma_k. \quad (7)$$

Each comparison has an absolute 10^{-12} momentum tolerance. Each individually empty window adds its nearest available mode; ties select the first mode in the sorted array. A zero-radius window therefore still retains a nearest mode without an exact resonance. Factors must be finite and nonnegative. Selection can yield asymmetry, holes, an even selected count, or no zero mode. These follow from the requested windows, not a hidden modification of the base-grid convention.

The packet is then rebuilt and normalized on the selected grid. A comparison therefore tests both preparation and dynamics. For a selected subset $S \subseteq \mathcal{K}$, with c_m the normalized unselected packet,

$$p_S = \sum_{m \in S} |c_m|^2, \quad c_m^{(S)} = c_m / \sqrt{p_S} \quad (m \in S), \quad F_{\text{state}}(0) = p_S, \quad F_{\text{atom}}(0) = 1 \quad (8)$$

for identical number preparations with $1 \leq n \leq N$, defined below. For a coherent preparation, renormalizing the selected packet also changes its individual mode amplitudes. The fixed-product projection formula below does not automatically apply across such different grids.

Three photon bases and their ordering. A photon configuration is $\mathbf{n} \in \mathbb{N}^M$. With $N = \mathbf{n_max}$,

$$\mathcal{B}_{\text{truncated}} = \{\mathbf{0}\} \cup \{n\mathbf{e}_m : 1 \leq m \leq M, 1 \leq n \leq N\}, \quad (9)$$

$$\mathcal{B}_{\text{total}} = \{\mathbf{n} : \sum_m n_m \leq N\}, \quad (10)$$

$$\mathcal{B}_{\text{full}} = \{\mathbf{n} : 0 \leq n_m \leq N \quad \forall m\}. \quad (11)$$

The string names are `truncated`, `full+totalcap` and `full`. The TLS is a separate $\mathbb{C}^2$ factor in all three. There is no supplemental atomic oscillator or `truncated+atom` basis. At fixed grid and cap,

$$\mathcal{B}_{\text{truncated}} \subseteq \mathcal{B}_{\text{total}} \subseteq \mathcal{B}_{\text{full}}. \quad (12)$$

The joint ket dimensions are

$$d_{\text{truncated}} = 2(1 + MN), \quad d_{\text{total}} = 2 \binom{M+N}{N}, \quad d_{\text{full}} = 2(N+1)^M. \quad (13)$$

Configurations with total r are weak compositions of r into M entries, numbering $\binom{M+r-1}{r}$. Summing over $0 \leq r \leq N$ gives $\binom{M+N}{N}$. At $N = 0$ every basis has one vacuum and two TLS components; at $N = 1$ truncated and total-cap coincide. For $M = 1$ all three coincide.

Indices are representation dependent. For $M = 2, N = 2$, the exact source order is:

i	<code>truncated</code>	<code>full+totalcap</code>	<code>full</code>
0	(0, 0)	(0, 0)	(0, 0)
1	(1, 0)	(1, 0)	(0, 1)
2	(2, 0)	(0, 1)	(0, 2)
3	(0, 1)	(2, 0)	(1, 0)
4	(0, 2)	(1, 1)	(1, 1)
5	—	(0, 2)	(1, 2)
6	—	—	(2, 0)
7	—	—	(2, 1)
8	—	—	(2, 2)

Total-cap enumerates combinations of mode indices with replacement and converts their multiplicities to occupations. Full uses a Cartesian product with the last entry changing fastest. In every representation, $s = 0$ denotes g , $s = 1$ denotes e , and

$$q = 2i + s, \quad v_{2i} = C_{i,g}, \quad v_{2i+1} = C_{i,e}, \quad |\psi\rangle = \sum_{i,s} C_{i,s} |\mathbf{n}_i; s\rangle. \quad (14)$$

An occupation-to-index dictionary supplies targets. Equal raw indices in different bases do not generally mean equal physical configurations.

1.2 Preparation, projected Hamiltonian and evolution

Gaussian coefficients and number preparation. For $k_0, x_0 \in \mathbb{R}$ and $\sigma_k > 0$,

$$\tilde{c}_m = \exp\left[-\frac{(k_m - k_0)^2}{4\sigma_k^2}\right] e^{-ik_m x_0}, \quad c_m = \frac{\tilde{c}_m}{\sqrt{\sum_\ell |\tilde{c}_\ell|^2}}. \quad (15)$$

All selected signed modes enter. There is no right-moving-only option: positive k_0 specifies a centre, not a filter. In the continuous ideal Gaussian, $|\tilde{c}(k)|^2$ has standard deviation σ_k . The actual finite-band moments are

$$\langle k \rangle_c = \sum_m k_m |c_m|^2, \quad (\Delta k_c)^2 = \sum_m (k_m - \langle k \rangle_c)^2 |c_m|^2, \quad (16)$$

and need not equal k_0 and σ_k^2 . The code subtracts the maximum real exponent before exponentiation. This common positive scale cancels at normalization and avoids an entirely underflowed packet for a distant centre.

The TLS preparation is one of

$$|a_0\rangle \in \left\{ |g\rangle, |e\rangle, (|g\rangle + |e\rangle)/\sqrt{2}, (|g\rangle - |e\rangle)/\sqrt{2} \right\}, \quad (17)$$

encoded by `initial_state='g', 'e', '+'` or `'-'` in the atomic dictionary, with default g . For `state='number'`,

$$|\psi_0^{(n)}\rangle = \begin{cases} |0\rangle \otimes |a_0\rangle, & n = 0, \\ \sum_m c_m |n\mathbf{e}_m\rangle \otimes |a_0\rangle, & 1 \leq n \leq N. \end{cases} \quad (18)$$

The default is this number preparation with $n = 1$. Orthogonality of the configurations proves normalization for $n > 0$. At $n = 0$ the vacuum is constructed once; summing it once per mode would duplicate one physical state.

For $n > 1$, all n photons occupy one mode at a time. A Fock state of a collective packet mode $b^\dagger = \sum_m c_m a_m^\dagger$ would instead be

$$\frac{(b^\dagger)^n}{\sqrt{n!}} |0\rangle = \sum_{\sum_m n_m = n} \sqrt{\frac{n!}{\prod_m n_m!}} \prod_m c_m^{n_m} |\mathbf{n}\rangle. \quad (19)$$

For two modes and $n = 2$, this has amplitudes $c_1^2, \sqrt{2}c_1c_2, c_2^2$ on $(2, 0), (1, 1), (0, 2)$; the implementation has c_1, c_2 on $(2, 0), (0, 2)$ only. The historical preparation is preserved, so higher- n results must use its actual definition.

Coherent projection and its retained weight. For `state='coherent'`, α is finite and complex, and

$$|\alpha\rangle = \bigotimes_m |\alpha c_m\rangle, \quad b_n = e^{-|\alpha|^2/2} \prod_m \frac{(\alpha c_m)^{n_m}}{\sqrt{n_m!}}. \quad (20)$$

The total unprojected mean photon number is $\lambda = |\alpha|^2$, with individual means $\lambda_m = \lambda |c_m|^2$ and $\sum_m \lambda_m = \lambda$. Let

$$P_{\mathcal{B}} = \sum_{\mathbf{n} \in \mathcal{B}, s} |\mathbf{n}; s\rangle \langle \mathbf{n}; s|, \quad Z_{\mathcal{B}} = \sum_{\mathbf{n} \in \mathcal{B}} |b_{\mathbf{n}}|^2. \quad (21)$$

The actual prepared ket is

$$|\psi_{0,\mathcal{B}}\rangle = \frac{P_{\mathcal{B}}(|\alpha\rangle \otimes |a_0\rangle)}{\sqrt{Z_{\mathcal{B}}}}. \quad (22)$$

The common exponential prefactor is not evaluated in the source because it cancels on normalization. With $E_N(z) = \sum_{r=0}^N z^r / r!$,

$$Z_{\text{full}} = e^{-\lambda} \prod_m E_N(\lambda_m), \quad (23)$$

$$Z_{\text{total}} = e^{-\lambda} E_N(\lambda), \quad (24)$$

$$Z_{\text{truncated}} = e^{-\lambda} \left[1 + \sum_m (E_N(\lambda_m) - 1) \right]. \quad (25)$$

The total-cap formula follows from the multinomial identity

$$\sum_{\sum_m n_m = r} \prod_m \frac{\lambda_m^{n_m}}{n_m!} = \frac{(\sum_m \lambda_m)^r}{r!} = \frac{\lambda^r}{r!}. \quad (26)$$

Truncated counts vacuum once and nonzero occupations on each mode ray. Consequently $0 < Z_{\text{truncated}} \leq Z_{\text{total}} \leq Z_{\text{full}} \leq 1$. At $\alpha = 0$ all weights equal one. If several λ_m are nonzero, increasing N in truncated alone approaches $e^{-\lambda}[1 + \sum_m (e^{\lambda_m} - 1)]$, generally below one: multimode occupations remain omitted at every cap.

For two projections of the *same* coherent product on the same grid,

$$F_{\text{state}}(0) = \frac{Z_{A \cap B}^2}{Z_A Z_B}, \quad Z_{A \cap B} = \sum_{\mathbf{n} \in \mathcal{B}_A \cap \mathcal{B}_B} |b_{\mathbf{n}}|^2, \quad F_{\text{atom}}(0) = 1. \quad (27)$$

If $\mathcal{B}_A \subseteq \mathcal{B}_B$, this is Z_A/Z_B . For two equally weighted modes, $\lambda = 1$ and $N = 2$, the weights are $2.25/e$, $2.5/e$, and $2.640625/e$ for truncated, total-cap and full. Truncated versus total-cap therefore starts at $F_{\text{state}}(0) = 0.9$ before any interaction. All these preparations are field-TLS products, explaining their unit atomic fidelity and zero initial atomic entropy.

Hamiltonian and matrix entries. For finite real D , $\omega_0 \geq 0$ and finite x_{tls} ,

$$H_0 = \sum_m |k_m| a_m^\dagger a_m + \omega_0 |e\rangle \langle e|, \quad (28)$$

$$u_m = \frac{if_m e^{-ik_m x_{\text{tls}}}}{\sqrt{L}}, \quad f_m = \begin{cases} \sqrt{|k_m|}, & \text{sqrt,} \\ 1, & \text{flat,} \end{cases} \quad (29)$$

$$H_{\mathcal{B}} = P_{\mathcal{B}} \left[H_0 + D \sum_m (u_m a_m + u_m^* a_m^\dagger) (\sigma_+ + \sigma_-) \right] P_{\mathcal{B}} \equiv H_{0,\mathcal{B}} + DV_{\mathcal{B}}, \quad (30)$$

where $\sigma_+ = |e\rangle \langle g|$ and $\sigma_- = |g\rangle \langle e|$. The factor i remains at zero TLS position. Creation receives the conjugate annihilation coefficient. Because field and TLS operators commute, $(u_m a_m \sigma_x)^\dagger = u_m^* a_m^\dagger \sigma_x$; the sum is Hermitian, and orthogonal projection preserves this property. In energy units, D is dimensionless for **sqrt** and has units $\sqrt{\text{energy}}$ for **flat**. Equal numerical couplings under the profiles do not describe the same physical model.

For a source column $q = 2i + s$, the assembled entries are

$$(H_0)_{2i+s, 2i+s} = \sum_m |k_m| n_{i,m} + s\omega_0, \quad (31)$$

$$(V_{\mathcal{B}})_{2j+1-s, 2i+s}^{(-)} = u_m \sqrt{n_{i,m}}, \quad \mathbf{n}_j = \mathbf{n}_i - \mathbf{e}_m, \quad (32)$$

$$(V_{\mathcal{B}})_{2j+1-s, 2i+s}^{(+)} = u_m^* \sqrt{n_{i,m} + 1}, \quad \mathbf{n}_j = \mathbf{n}_i + \mathbf{e}_m \in \mathcal{B}. \quad (33)$$

A descending transition needs $n_{i,m} > 0$. All bases are downward closed, so the target then exists. An ascending target is looked up and omitted if absent, realizing exactly $P_{\mathcal{B}} H P_{\mathcal{B}}$. No omitted target is replaced by another oscillator. A transition and its reverse are both retained or both absent, with conjugate coefficients.

For example, creation in mode two from $(1, 0)$ targets $(1, 1)$: retained in total-cap/full at $N = 2$, absent in truncated. Creation from $(2, 0)$ into $(2, 1)$ is retained only in full. Shared entries have identical coefficients, but different transition graphs can produce different dynamics. Projected ladder operators are not canonical at the boundary. For one mode,

$$a_N |n\rangle = \sqrt{n} |n-1\rangle, \quad a_N^\dagger |N\rangle = 0, \quad [a_N, a_N^\dagger] = I - (N+1) |N\rangle \langle N|. \quad (34)$$

A calculation can be exact for this finite model without proving convergence to unbounded Fock space. Assembly costs $O(Md)$, rather than scanning all pairs.

RWA, parity and the zero mode. Under RWA only annihilation with excitation and creation with de-excitation remain:

$$H_{\mathcal{B}, \text{RWA}} = P_{\mathcal{B}} \left[H_0 + D \sum_m (u_m a_m \sigma_+ + u_m^* a_m^\dagger \sigma_-) \right] P_{\mathcal{B}}. \quad (35)$$

With $\mathcal{N}_f = \sum_m n_m$, $\mathcal{N} = \mathcal{N}_f + |e\rangle \langle e|$ and $\Pi = (-1)^{\mathcal{N}}$,

$$[H_{\mathcal{B}, \text{RWA}}, \mathcal{N}] = 0, \quad [H_{\mathcal{B}}, \Pi] = 0. \quad (36)$$

RWA transitions change photon and atomic excitation by opposite units. Counter-rotating terms change their sum by ± 2 , preserving its parity. Occupation projectors commute with the diagonal symmetry operators, so these identities survive all three projections.

For the retained zero mode and **sqrt**,

$$\omega(0) = f(0) = 0, \quad [H_{\mathcal{B}}, n_0] = 0. \quad (37)$$

It can carry initial norm without free energy or direct TLS coupling. In full, fixing $n_0 = r$ leaves the same nonzero-mode Hamiltonian for each r . In total-cap the corresponding sector instead obeys

$$\sum_{m: k_m \neq 0} n_m \leq N - r. \quad (38)$$

A conserved zero-mode occupation thus changes the available nonzero photon space. In truncated, positive zero-mode occupation cannot coexist with occupation of another mode. Deleting zero changes preparation normalization and these finite spaces even though $f(0) = 0$. For flat, $f(0) = 1$ and the zero mode couples, so n_0 is generally no longer conserved. At $N = 0$ all interactions are projected away. At explicit $M = 1$ with sqrt the only mode is zero and only TLS free evolution remains.

Propagation, output sampling and numerical normalization. The engine solves the time-independent equation

$$i \frac{d}{dt} |\psi(t)\rangle = H_B |\psi(t)\rangle, \quad |\psi(t)\rangle = e^{-iH_B t} |\psi_0\rangle. \quad (39)$$

The default integration method used by this engine is BDF. Relative and absolute tolerances are `rtol=1e-9` and `atol=1e-11`. Output renormalization is disabled (`normalize_output=False`). Output `dt` is not the adaptive solver's internal step. Nominal outputs are $0, dt, \dots, Jdt$, $J = \lfloor T/dt \rfloor$, with T appended if needed. Endpoint roundoff is handled at 10^{-12} ; both zero and exact T are always included. If `dt` exceeds T , the list is $[0, T]$.

Exact evolution conserves $\|v\|^2$ and $\langle H_B \rangle$. The reported norm is the squared norm, not its square root. Coefficients, populations and energies are not renormalized after integration. Entropy and fidelity normalize internally. If $v_{\text{num}} = (1 + \epsilon)e^{i\theta}v_{\text{exact}}$, its fidelity is one after normalization but its raw norm is $|1 + \epsilon|^2$. Fidelity alone therefore does not certify norm accuracy.

With `store_state=True`, solver history and coefficient arrays contain all outputs. With `store_state=False`, only the final ket and final coefficient arrays are retained; the output-time list still exists. Time-series fidelity requires both histories. Diagnostic methods accept `t=None` for all retained states, `t=-1` for final, or a physical time in $[0, T]$. With history, a time selects the nearest output without interpolation; a final-only run rejects earlier times. The wavefunction method instead takes an output *index*.

1.3 Observables, exact fidelities and physical interpretation

TLS reduction and entanglement. Tracing equation (14) over field configurations gives

$$\rho_a = C^\top C^*, \quad (\rho_a)_{ss'} = \sum_i C_{i,s} C_{i,s'}^*, \quad P_e = \sum_i |C_{i,e}|^2, \quad \text{Tr } \rho_a = \|v\|^2. \quad (40)$$

The transpose is intentionally not an adjoint: rows of C are photon configurations and columns are TLS labels. For $\hat{\rho}_a = \rho_a / \|v\|^2 = \begin{pmatrix} p_g & z \\ z^* & p_e \end{pmatrix}$,

$$\lambda_{\pm} = \frac{1 \pm \sqrt{(p_g - p_e)^2 + 4|z|^2}}{2}, \quad S_a = - \sum_{r=\pm} \lambda_r \log \lambda_r. \quad (41)$$

The logarithm is natural and $0 \log 0 = 0$, so $0 \leq S_a \leq \log 2$. For a global pure ket this measures field-TLS entanglement: zero for a product, maximal for a maximally mixed TLS reduction. The excited-probability method returns the raw diagonal of ρ_a , exposing normalization drift.

Photon sectors and Fourier reconstruction. Directional populations refer specifically to one photon and ground-state TLS:

$$P_{+,1g} = \sum_{m:k_m > 0} |C_{i(e_m),g}|^2, \quad P_{-,1g} = \sum_{m:k_m < 0} |C_{i(e_m),g}|^2, \quad (42)$$

$$P_{0,1g} = \sum_{m:k_m = 0} |C_{i(e_m),g}|^2, \quad p_\nu = \sum_{i: \sum_m n_{i,m} = \nu} \sum_s |C_{i,s}|^2. \quad (43)$$

Their keys are `p_transmitted`, `p_reflected`, `p_zero_1g` and `p_<nu>_photon`. Exact sum rules are

$$P_{+,1g} + P_{-,1g} + P_{0,1g} = P_{1g} \leq p_1, \quad \sum_\nu p_\nu = \|v\|^2. \quad (44)$$

The difference $p_1 - P_{1g}$ is the one-photon, excited-TLS population. Photon totals range through MN in full and N in the other two bases. Directional populations vanish at $N = 0$.

The extracted function is

$$\phi_{1g}(x, t) = \frac{1}{\sqrt{L}} \sum_m C_{i(e_m),g}(t) e^{ik_m x}, \quad \int_{x_*}^{x_*+L} |\phi_{1g}(x, t)|^2 dx = P_{1g}(t). \quad (45)$$

This is a component of the ket, not a density of all photons in a multiphoton state. Its integrated weight may be less than one. The integral uses orthogonality from equation (1); a plotting mesh approximates it. The function is periodic. Under free evolution a positive-momentum component obeys $\phi(x, t) = \phi(x - t, 0)$ and a negative-momentum component obeys $\phi(x, t) = \phi(x + t, 0)$. Both directions can be present in the packet.

The imposed spatial phases give, for an input in the $1g$ sector,

$$(H\psi)_{\mathbf{0},e} = \frac{iD}{\sqrt{L}} \sum_m f_m C_{i(e_m),g} e^{-ik_m x_{\text{tls}}}, \quad (46)$$

$$\text{for flat: } (H\psi)_{\mathbf{0},e} = iD \phi_{1g}(-x_{\text{tls}}). \quad (47)$$

For sqrt the same coordinate enters the field weighted by $\sqrt{|k_m|}$. The scattering plot therefore marks $-x_{\text{tls}}$ in its positive-Fourier coordinate. The parameter remains the one in the requested annihilation phase. Identifying it directly with the plotted coordinate would require the opposite annihilation-phase sign; at zero they coincide. A general non-RWA input can have additional contributions to this target from other sectors.

Total Hamiltonian expectation and energy conservation. For the joint coefficient vector $v_q(t) = C_{i,s}(t)$, $q = 2i + s$, and matrix $H_{rc} = \langle r | H_{\mathcal{B}} | c \rangle$, the total energy of an exactly normalized state is

$$E(t) = \langle \psi(t) | H_{\mathcal{B}} | \psi(t) \rangle = v(t)^\dagger H_{\mathcal{B}} v(t) = \sum_{r,c} v_r^*(t) H_{rc} v_c(t). \quad (48)$$

Here r, c label joint field-TLS basis states. Since $H_{\mathcal{B}}$ is Hermitian, the sum is real. With $\dot{\psi} = -iH_{\mathcal{B}}\psi$ and $\partial_t H_{\mathcal{B}} = 0$,

$$\frac{dE}{dt} = \langle \dot{\psi} | H_{\mathcal{B}} | \psi \rangle + \langle \psi | H_{\mathcal{B}} | \dot{\psi} \rangle = i \langle \psi | H_{\mathcal{B}}^2 | \psi \rangle - i \langle \psi | H_{\mathcal{B}}^2 | \psi \rangle = 0. \quad (49)$$

Thus $E(t) = E(0)$ for exact evolution in each finite model. Energy conservation is a numerical diagnostic, not a certificate of full-state convergence.

The method `compute_energy` returns the raw quadratic expression, without renormalizing propagated kets. If the numerical squared norm is $\eta(t) = v(t)^\dagger v(t)$, distinguish

$$E_{\text{raw}}(t) = \Re[v(t)^\dagger H_{\mathcal{B}} v(t)], \quad E_{\text{normalized}}(t) = E_{\text{raw}}(t)/\eta(t) \quad (\eta > 0). \quad (50)$$

They coincide when $\eta = 1$. For an already propagated experiment:

```
1 observables = experiment.compute_observables()
2 time = observables['time']
3 E_raw = experiment.compute_energy()
4 E_normalized = E_raw / observables['norm']
```

The returned energy array spans the retained outputs. An explicit time selects the nearest stored output; a final-only run returns its final value. Monitoring $\max_j |E_{\text{raw}}(t_j) - E_{\text{raw}}(0)|$ alongside the norm and solver tolerances tests conservation without dividing by a possibly zero initial energy.

Implicit embedding in a common full space. Comparisons require a common physical box and consistent conventions. Let $\mathcal{K}_* = \mathcal{K}_A \cup \mathcal{K}_B$ and $N_* \geq \max(N_A, N_B)$; the ambient space is

$$\mathcal{H}_* = \text{span}\{|\mathbf{n}; s\rangle : 0 \leq n_m \leq N_*, k_m \in \mathcal{K}_*, s = g, e\}. \quad (51)$$

A physical mode map μ_X defines

$$(\iota_X \mathbf{n})_{\mu_X(m)} = n_m, \quad (\iota_X \mathbf{n})_\ell = 0 \text{ for absent modes}, \quad \iota_X |\mathbf{n}; s\rangle = |\iota_X \mathbf{n}; s\rangle. \quad (52)$$

Distinct tuples stay orthogonal, so

$$\iota_X^\dagger \iota_X = I_X, \quad \|\iota_X \psi_X\| = \|\psi_X\|. \quad (53)$$

For example, adding $k = 0$ maps $|n_{-1}, n_{+1}; s\rangle$ to $|n_{-1}, 0, n_{+1}; s\rangle$. Positive occupation on that added mode is orthogonal to every such embedded ket. Reordering existing modes only permutes coordinates and cannot change an overlap.

For individually normalized kets,

$$A_{AB} = \langle \iota_A \psi_A | \iota_B \psi_B \rangle = \sum_{\mathbf{n} \in \iota_A \mathcal{B}_A \cap \iota_B \mathcal{B}_B} \sum_s C_{A,\mathbf{n},s}^* C_{B,\mathbf{n},s}, \quad (54)$$

$$F_{\text{state}} = |A_{AB}|^2. \quad (55)$$

The intersection enters because missing amplitudes are zero. No projection and renormalization on the intersection is performed. For orthogonal configurations x, y , take $|\psi_A\rangle = |x\rangle$ and $|\psi_B\rangle = \sqrt{p}|x\rangle + \sqrt{1-p}|y\rangle$. Then

$$F_{\text{state}} = p; \quad \text{renormalizing the common component would incorrectly give 1.} \quad (56)$$

For distinct one-photon modes a, b , the states $(|0; g\rangle + |1_a; g\rangle)/\sqrt{2}$ and $(|0; g\rangle + |1_b; g\rangle)/\sqrt{2}$ share only vacuum. Their overlap is 1/2 and their state fidelity is 1/4.

Keys contain sorted pairs (physical mode index, n_m) for $n_m > 0$; vacuum has the empty key. Each stores two TLS amplitudes. All shared configurations contribute, including several occupied modes. No vector of dimension $2(N_* + 1)^{|\mathcal{K}_*|}$ is created. The union retains the first grid's order and appends absent modes; sorting keys makes that internal order irrelevant. Momentum matching uses absolute tolerance 10^{-12} , zero relative tolerance, and rejects ambiguous identifications.

The time-series interface checks equal L to absolute tolerance 10^{-12} and exactly equal output arrays before pairing states. Low-level `compare_states` has no L or Hamiltonian input, so its caller must ensure physical comparability. Matching numerical momenta from different boxes alone does not certify identical normalized modes.

Atomic fidelity, normalization and bounds. The atomic metric is squared Uhlmann fidelity,

$$F_{\text{atom}} = \left[\text{Tr} \sqrt{\sqrt{\hat{\rho}_{a,A}} \hat{\rho}_{a,B} \sqrt{\hat{\rho}_{a,A}}} \right]^2. \quad (57)$$

The code squares the value returned by `qutip.fidelity`. For qubits,

$$F_{\text{atom}} = \text{Tr}(\hat{\rho}_{a,A} \hat{\rho}_{a,B}) + 2\sqrt{\det \hat{\rho}_{a,A} \det \hat{\rho}_{a,B}}. \quad (58)$$

To derive this, let $X = \sqrt{\rho} \sigma \sqrt{\rho}$ have eigenvalues x_1, x_2 . Then $(\text{Tr} \sqrt{X})^2 = x_1 + x_2 + 2\sqrt{x_1 x_2}$, $\text{Tr} X = \text{Tr}(\rho \sigma)$ and $\det X = \det \rho \det \sigma$. Rank-deficient cases follow by continuity. For Bloch vectors $\mathbf{r}, \mathbf{s}$,

$$F_{\text{atom}} = \frac{1 + \mathbf{r} \cdot \mathbf{s} + \sqrt{(1 - |\mathbf{r}|^2)(1 - |\mathbf{s}|^2)}}{2}. \quad (59)$$

It reduces to squared overlap for pure atomic states, and against a pure $|e\rangle$ to the normalized excited population. Adding vacuum modes leaves the atomic reduction invariant; no ambient allocation is required.

Both metrics normalize each entire finite nonzero ket, not a common subset. Values are clipped to $[0, 1]$ for roundoff overshoots. Raw norm diagnostics remain visible. Monotonicity under the common partial trace yields

$$0 \leq F_{\text{state}} \leq F_{\text{atom}} \leq 1. \quad (60)$$

For instance, $|1_+; g\rangle$ and $|1_-; g\rangle$ have state fidelity zero and atomic fidelity one. No ratio is calculated: a quotient of these metrics is not generally a fidelity of the field reductions.

For normalized embedded pure states, $\rho_A - \rho_B$ has nonzero eigenvalues $\pm \sqrt{1 - |\langle \psi_A | \psi_B \rangle|^2}$ in their two-dimensional span. Therefore, for any common projector, or more generally $0 \leq Q \leq I$,

$$|\text{Tr}[Q(\rho_A - \rho_B)]| \leq \frac{1}{2} \|\rho_A - \rho_B\|_1 = \sqrt{1 - F_{\text{state}}}. \quad (61)$$

This bounds disagreement between normalized populations of two finite calculations, not accuracy against a continuum experiment. For TLS-only measurements an analogous upper bound uses $\sqrt{1 - F_{\text{atom}}}$.

Sample means and scattering interpretation. The coupling sweep and mode selection report

$$\bar{F}_{\text{samples}} = \frac{1}{N_t} \sum_{j=0}^{N_t-1} F(t_j). \quad (62)$$

This differs from final fidelity and a time integral. A trapezoidal time average would instead be

$$\bar{F}_{\text{time}} \simeq \frac{1}{T} \sum_{j=0}^{N_t-2} \frac{F(t_j) + F(t_{j+1})}{2} (t_{j+1} - t_j). \quad (63)$$

Only equation (62) is implemented. Endpoints have different weights even on a uniform grid; a shorter final interval still has an equally weighted output in the sample mean. Initial metrics are stored separately. Cap convergence compares final kets for adjacent entries in the supplied increasing cap list, which need not differ by one.

Scattering supplies finite-packet sector populations and Fourier profiles. If $\sum_{k_m < 0} |c_m|^2 > 0$, final negative-momentum population is not by itself a reflection probability for an exclusively positive-momentum input. Subtracting the initial population need not remove interference. For RWA evolution from one photon and ground TLS, excitation conservation gives

$$P_{+,1g} + P_{-,1g} + P_{0,1g} + P_e = 1 \quad (64)$$

for a normalized state. Without RWA additional sectors can carry norm, so this four-term identity is not general.

Extracting $t(k)$ or $r(k)$ needs identifiable outgoing packets, control of periodic returns and a phase convention relative to free propagation. Counter-rotating terms can require dressed asymptotic states and additional photon channels. The calculated quantities remain finite-time projections; neither fidelity is an S -matrix element.

2 Notebook control, resources and source-line commentary

2.1 Shared configuration and callable Python experiments

Configuration and execution. `ExperimentConfig` collects three physical dictionaries, cutoffs, cap, basis name, RWA, storage choice, grid control and selection windows. Validation occurs when resolving the grid, building the basis/Hamiltonian and preparing the ket. `with_D` returns a copy with a new atomic dictionary. An `Experiment` copies its physical dictionaries before construction, so sweeps do not overwrite the supplied physical configuration. A notebook parameter cell can use:

```

1 import numpy as np
2 from dataclasses import replace
3 from src.xp_config import ExperimentConfig
4 from src.experiment import estimate_config
5 from src.fidelities import fidelities_over_time
6 from experiment.atom_evolution import (
7     run_atom_evolution, plot_atom_evolution,
8 )
9
10 config = ExperimentConfig(
11     param_photon={'k_0': 1., 'sigma_k': .4, 'x_0': -2.,
12                  'state': 'number', 'n': 1},
13     param_atom={'omega_0': 1., 'D': .2, 'L': 2*np.pi,
14                'x_tls': 0., 'coupling': 'sqrt',
15                'initial_state': 'g'},
16     param_time_evol={'T': 4., 'dt': .05,
17                     'rtol': 1e-9, 'atol': 1e-11},
18     n_max=2, CTRL_M_EXPLICIT=True, M=5,
19     truncation='full+totalcap', store_state=True)
20
21 # Estimate every basis that will actually be propagated.
22 for scheme in ('full+totalcap', 'truncated'):
23     estimate_config(replace(config, truncation=scheme))
24 runs = run_atom_evolution(config)
25 metrics = fidelities_over_time(
26     runs['truncated'], runs['full+totalcap'])
27 fig = plot_atom_evolution(runs)

```

For cutoff control, use `CTRL_M_EXPLICIT=False` and `cutoffs={'ir_cutoff': 0., 'uv_cutoff': 2.}` instead. At this L the base grid is identical. Direct execution constructs `Experiment`, calls `propagate_state`, then `compute_observables`; the common experiment helper performs those three operations.

Five experiments and their output contracts. All four migrated Uri notebooks explicitly set `coupling='sqrt'`. The scattering notebook remains alongside them. Modules do not simulate or change the Matplotlib backend on import. Numerical execution and plotting are separate functions, with plotting returning figure objects.

Module	Calculation and main return value
<code>atom_evolution.py</code>	Dictionary of propagated <code>Experiment</code> objects by requested basis. Forces history storage; plots P_e and entropy. The notebook additionally calculates the two fidelities.
<code>full_cumulative_fidelity.py</code>	Dictionary of increasing caps, retained final-only runs, and initial/final metrics for adjacent cap-list entries. Always uses total-cap.
<code>sweep_D_fidelity.py</code>	Dictionary of physical couplings, candidate names, comparator name, sample means and initial metrics. Forces history.
<code>compare_mode_selection.py</code>	Coupling sweeps with/without selection, and window heatmaps at fixed coupling. Requires history. Comparator is always unselected total-cap.
<code>scattering.py</code>	One experiment with the user's dictionary API; optional observable CSV. Plots $1g$ profiles and directional/zero/TLS/norm populations.

With S candidate schemes, Q coupling values, C caps, and P, A window factor counts, the layouts are:

Result	Array shapes and axes
Atom evolution	Ground/excited coefficient arrays (N_t, B) ; population, entropy and time-series fidelity arrays $(N_t,)$.
Cap convergence	Each metric and its <code>_initial</code> counterpart: $(C - 1,)$. Entry j compares supplied caps j and $j + 1$.
Coupling sweep	Each metric and its <code>_initial</code> counterpart: (S, Q) , scheme before coupling. Unsuffix metrics are sample means.
Selection	Sweep metrics $(S, 2, Q)$, with middle axis <code>[False, True]</code> . Each <code>_heatmap</code> : (S, P, A) , photon window before atomic window.
Scattering	Observable arrays of length N_t with history, one otherwise; wavefunction array indexed by the supplied spatial samples.

For final-only runs the coefficient arrays have shape $(B,)$, not $(1, B)$; observable methods promote them internally as needed. Diagnostics with `t=None` return a length-one array/list then, whereas an explicit final time returns one scalar or matrix.

Cap convergence compares equal physical/grid inputs with different caps. Its default CLI preparation is coherent, so initial projection differences are expected. The coupling sweep defaults to a propagated total-cap comparator; `reference` can select another finite basis. This is distinct from the ambient full space in equation (51). A candidate with the comparator's scheme reuses its trajectory.

The selection study rebuilds an unselected comparator at each D . Heatmaps use `D_heatmap` when given, otherwise the input configuration's coupling. Their returns contain only the sample means, not separate initial heatmap metrics; the sweep arrays do contain the `_initial` values. This distinction matters when separating selected-preparation and dynamical effects.

Command-line execution, exports and notebook state. The five experiments are invoked through their modules in `experiment/`. Shared arguments use `-option value`; old numerical values still require the convention conversions given in the comparative analysis. From the repository root:

```

1 python -m experiment.scattering --D 0.2 --ctrl-m-explicit --M 5
2 python -m experiment.sweep_D_fidelity --D-values 0.02,0.1,0.2 \
3   --out results/sweep.png
4 python -m experiment.atom_evolution --D 0.2 --out results/atom.png

```

The CLI defaults to cutoff control; `-M` is active only with `-ctrl-m-explicit`. Both profiles are available, with default `sqrt`; `-RWA` enables RWA. Solver options can be set in the Python time dictionary. Notebooks resolve the root whether the kernel starts there or in `notebooks/`. Markdown, comments, labels and function docstrings are in English. Each function defines the meaning, types and array shapes of its inputs and outputs, with formulas for the relevant numerical operations.

Resource and diagnostic outputs are stored in pandas DataFrames rather than line-by-line prints. Numerical estimate keys remain available alongside `grid_table` and `resource_table`; setting `print_report=False` suppresses automatic display without discarding either table. The notebooks keep named DataFrames for reuse or export:

```

1 estimate = estimate_config(config, print_report=False)
2 grid_table = estimate['grid_table']
3 resource_table = estimate['resource_table']
4 display(grid_table, resource_table)
5
6 # After propagation:
7 observable_history = experiment.observables_dataframe()
8 observable_summary = experiment.observables_dataframe(summary=True)
9 display(observable_summary)

```

The history has physical time as its index and raw observables as columns. The summary has observable names as rows and `initial/final` columns, including a row for time. Final-only storage produces just a `final` column: its retained state is never presented as an initial sample. Neither conversion normalizes populations nor rounds stored values. Fidelity DataFrames expose initial, final or sample-mean metrics with explicit labels; sweeps use indices naming the scheme, coupling, selection flag or compared caps. Window scans use scheme and both window radii as index levels. DataFrames support their usual CSV export. The estimator records excluded memory costs in `resource_table.attrs['memory_excludes']` and the heuristic feasibility rule in `resource_table.attrs['feasibility_rule']`.

With `-out`, a CLI saves a figure and same-stem NPZ with the supplied arrays. Its metadata field is `configuration_json`. JSON stores the base configuration; additional sweep choices and derived grids must be read from accompanying arrays or reconstructed. Complex values are represented as strings. Ordinary exported arrays require no pickle:

```
1 import json
2 with np.load('results/sweep.npz', allow_pickle=False) as data:
3     metadata = json.loads(str(data['configuration_json']))
4     D_values = data['D']
5     F_state_mean = data['F_state']
```

The atom-evolution CLI exports time and excited populations; entropy and notebook fidelity curves are not automatically included in that export. Cap exports omit Python run objects. Optional scattering CSV writes `results/scattering.csv`, overwriting a prior requested export, with one column per observable and no configuration metadata.

2.2 Correspondence between controls and resource costs

What the grid report establishes. The same `resolve_grid` serves propagation and estimation before Fock allocation. Base and selected descriptions contain counts, signed extrema, radial extrema, zero presence, smallest nonzero momentum and integer indices. For integer index set I , the report tests

$$I = \{j : -j_{\max} \leq j \leq j_{\max}, |j| \geq j_{\min}\}, \quad j_{\min} = \min_{j \in I} |j|, \quad j_{\max} = \max_{j \in I} |j|. \quad (65)$$

This is `radial_cutoffs_exact`. If also $j_{\min} = 0$, the count is `equivalent_M`; otherwise that field is `None`. Extrema alone cannot represent asymmetry or holes. At $L = 2\pi$:

Inputs	Grid	Radial exact	Equivalent M
$M = 5$	$\{-2, -1, 0, 1, 2\}$	Yes	5
IR/UV = (0, 2)	$\{-2, -1, 0, 1, 2\}$	Yes	5
IR/UV = (0.2, 2.8)	$\{-2, -1, 1, 2\}$	Yes	None
IR/UV = (0, 0)	$\{0\}$	Yes	1
Selected $\{-2, 0, 2\}$	$\{-2, 0, 2\}$	No	None
Selected $\{0, 1, 2\}$	$\{0, 1, 2\}$	No	None

The first two grids give identical finite dynamics at equal other inputs. A positive-IR band has no allowed equivalent explicit count. The report states this rather than proposing a count that silently inserts zero.

Vector storage and exact dimension growth. A complex128 ket occupies $16d$ bytes. With $S = \text{store_state}$,

$$G_{\text{ket}} = \frac{16d}{2^{30}}, \quad G_{\text{history}} = G_{\text{ket}} \begin{cases} N_t, & S, \\ 1, & \neg S, \end{cases} \quad (66)$$

$$G_{\text{vectors}} \simeq 2G_{\text{history}}, \quad \text{nnz}(H) \leq d(1 + 2M). \quad (67)$$

GiB means 2^{30} bytes. The factor two estimates solver kets and the contiguous coefficient array. Alternating ground/excited views share that array and do not add two more full copies. The initial ket, QuTiP objects, transient conversions and solver workspace can increase storage further.

For $M = 5, N = 2$, joint dimensions are 22, 42, 486 for truncated, total-cap and full. At $T = 4, \text{dt} = 0.05$, $N_t = 81$ and estimated retained vector bytes are 57 024, 108 864, and 1 259 712. At fixed N and large M ,

$$d_{\text{total}} \sim 2M^N/N!, \quad d_{\text{truncated}} \sim 2NM, \quad d_{\text{full}} = 2(N+1)^M. \quad (68)$$

Polynomial growth at fixed cap does not make high caps cheap; the exact binomial dimension should be evaluated before allocation.

Sparse transitions and excluded costs. Let $A_{\mathcal{B}}$ count photon edges $\mathbf{n} \leftrightarrow \mathbf{n} + \mathbf{e}_m$ in a basis, without TLS labels. For $N \geq 1$,

$$A_{\text{truncated}} = MN, \quad (69)$$

$$A_{\text{total}} = M \binom{M+N-1}{N-1}, \quad (70)$$

$$A_{\text{full}} = MN(N+1)^{M-1}. \quad (71)$$

For total-cap, remove one photon in the changed mode and count configurations with total at most $N-1$. For full, choose the mode, one of its N adjacent levels, and the other $M-1$ occupations. At $N=0$ all counts are zero. The engine assembles $4A_{\mathcal{B}}$ directed TLS-flipping triplets without RWA, $2A_{\mathcal{B}}$ with it. Zero coefficients, such as the sqrt zero mode, may still occupy recorded triplet slots. Physically nonzero matrix entries can be fewer, especially at $D=0$. The estimator uses the simpler bound above.

A complex CSR matrix with Z stored entries uses approximately

$$16Z + b_{\text{idx}}Z + b_{\text{idx}}(d+1) \quad \text{bytes} \quad (72)$$

for its numerical arrays, where b_{idx} depends on the sparse index type. The engine retains H_0 , V and its QuTiP Hamiltonian. Temporary row/column/value lists and COO/CSR conversion add assembly peaks. Basis tuples, dictionary entries and photon-number arrays have at least MB occupation entries plus object overhead. These costs are excluded from the returned vector estimate.

feasible is only the heuristic $d \leq 100000$ and $G_{\text{vectors}} \leq 1$ GiB. It is not a memory guarantee, a physical accuracy criterion or an automatic engine limit. Notebooks check it before allocation. Estimates concern one trajectory: retained objects at several caps/schemes add their costs. Cap convergence keeps all its runs, even though they are final-only. Sweep peaks include a comparator and candidate together. The largest individual estimate is therefore not a total collection estimate.

Implicit fidelity keys examine $O(M_A B_A + M_B B_B)$ occupation entries. Sorting each key adds work dependent on its occupied-mode count; matching physical modes searches the current union for each second-grid mode. Keys and dictionaries add memory, but no ambient full vector is allocated. Choosing full for an actual propagation still incurs its exponential dimension.

2.3 Numbered code and comments on individual operations

Listings below are loaded directly from executed sources. Commentary tables identify current line ranges and relate their operations to the equations. The generator finds ordered engine markers and syntax-tree function ranges for experiments; line numbers are not copied from old versions. Comments cover validations, tensor ordering, phases, missing transitions, normalization, output shapes and statistical choices. Core-engine ranges are deliberately narrower than experiment function ranges. To reproduce the documentation from the repository root:

```
1 python documentation/build_code_notes.py
2 latexmk -pdf -cd documentation/main.tex
```

These commands generate documentation only.

src/xp_config.py

```
1 """Shared configuration for scripts and notebooks."""
2
3 from dataclasses import dataclass, replace
4
5
6 @dataclass
7 class ExperimentConfig:
8     """Shared physical, basis, grid and solver inputs; no simulation on creation.
9
10     Parameters
11     -----
12     param_photon : dict[str, object]
13         k_0 (float): signed packet momentum centre; sigma_k (positive float):
14         Gaussian width in  $\exp(-(k-k_0)^2/(4*\sigma_k^2))$ ; x_0 (float):
15         initial spatial centre through  $\exp(-i*k*x_0)$ . state='number'/'coherent';
16         n (int) is the number-state photon count and alpha (complex) the
17         collective coherent amplitude before projection ( $\text{mean } |\alpha|^2$ ).
18     param_atom : dict[str, object]
19         omega_0 (float): TLS excitation energy above g; D (real float):
20         physical scale multiplying V in  $H=H_0+D*V$ ; L (positive float):
21         periodic box length; x_tls (float): TLS coordinate in  $\exp(-i*k*x_{\text{tls}})$ .
22         coupling='sqrt'/'flat' sets  $f_m=\sqrt{|k_m|}/1$ ; initial_state is one
23         of {'g','e','+', '-'}. Natural units  $\hbar=c=1$  are used throughout.
24     param_time_evol : dict[str, object]
25         T and dt (positive floats); optional method, rtol and atol. dt selects
26         outputs, while the solver chooses its internal integration steps.
27     cutoffs : dict[str, float] or None
28         ir_cutoff and uv_cutoff, active only when CTRL_M_EXPLICIT=False.
29     n_max : int
30         Nonnegative photon cutoff N, interpreted according to truncation.
31     truncation : str
32         'truncated', 'full+totalcap' (default), or 'full'.
33     RWA : bool
34         Select excitation-conserving coupling terms when True.
35     store_state : bool
36         True retains the history; False retains only the final state.
37     CTRL_M_EXPLICIT : bool
38         True chooses an odd positive M; False derives the count from IR/UV.
39     M : int or None
40         Explicit base-grid count, ignored under cutoff control.
41     mode_selection : bool
42         Apply packet/resonance windows after constructing the base grid.
43     photon_window, atom_window : float
44         Nonnegative window radii in units of sigma_k, default 1.5 and 0.25.
45
46     Notes
47     -----
48     Dataclass fields are available as attributes; consuming routines validate
49     them. replace(config, ...) makes a new dataclass, but dictionaries remain
50     shared unless explicitly replaced. with_D copies the atomic dictionary.
51     """
52     param_photon: dict
53     param_atom: dict
54     param_time_evol: dict
55     cutoffs: dict | None = None
56     n_max: int = 3
```

```

57 truncation: str = "full+totalcap"
58 RWA: bool = False
59 store_state: bool = True
60 CTRL_M_EXPLICIT: bool = False
61 M: int | None = None
62 mode_selection: bool = False
63 photon_window: float = 1.5
64 atom_window: float = 0.25
65
66 def with_D(self, D):
67     """Return a configuration with a new physical coupling without mutating input.
68
69     Parameters
70     -----
71     D : float
72         Coupling coefficient in  $H=H_0+D*V$ ; validated when H is constructed.
73
74     Returns
75     -----
76     ExperimentConfig
77         New dataclass with a copied param_atom dictionary containing D. Other
78         dictionaries remain shared by reference; this method does not run a
79         simulation or change the original configuration.
80     """
81     return replace(self, param_atom={**self.param_atom, "D": D})

```

Lines	Mathematical and implementation commentary
1–5	This module defines configuration data only. Imports do not allocate a grid or launch propagation.
6–51	The dataclass preserves named inputs and supports <code>dataclasses.replace</code> ; validation occurs in the consuming numerical routines.
52–52	Packet inputs k_0, σ_k, x_0 , preparation type, and either integer n or complex α ; see (15)–(22).
53–53	$\omega_0, D, L, x_{\text{tls}}$, coupling profile, and optional initial TLS label. The box length is stored here but also controls the photon grid.
54–54	Final time T , output spacing dt , and optional integration method/tolerances. Output spacing does not fix internal solver steps.
55–55	The radial band is required only under cutoff control. Explicit-count control ignores this dictionary.
56–59	N and the basis select the retained occupation set, (11). RWA selects Hamiltonian terms; storage selects retained outputs, not a different Hilbert space.
60–61	True selects exact odd M with zero, (2); false selects the radial band, (3). No hidden decrement of M occurs.
62–65	Additional mode windows, applied after resolving the base grid. Their radii are factors times σ_k , (7).
66–81	Return a replaced configuration with a new atomic dictionary. Other inputs are reused; the supplied dictionary is not overwritten during a sweep.

```

src/grid.py
1 """One momentum-grid resolver for evolution and resource estimation."""
2
3 from numbers import Integral
4
5 import numpy as np
6
7
8 def integer(value, name, minimum=0):
9     """Validate an integer input without silently rounding or accepting booleans.
10
11     Parameters
12     -----
13     value : int or numpy.integer
14         Candidate integer. Floats (including 2.0) and booleans are rejected.

```

```

15     name : str
16         Parameter name used in the validation error.
17     minimum : int, optional
18         Inclusive lower bound, default zero.
19
20     Returns
21     -----
22     int
23         Python integer equal to value, satisfying value >= minimum.
24
25     Raises
26     -----
27     ValueError
28         The input has the wrong type or lies below the bound.
29     """
30     if isinstance(value, bool) or not isinstance(value, Integral) or value < minimum:
31         raise ValueError(f"{name} must be an integer >= {minimum}")
32     return int(value)
33
34
35 def momentum_modes(param_atom, cutoffs=None, CTRL_M_EXPLICIT=False, M=None):
36     """Construct the signed periodic-box momenta  $k_j = j * (2\pi/L)$ .
37
38     Parameters
39     -----
40     param_atom : dict[str, object]
41         Contains finite positive box length 'L' (float).
42     cutoffs : dict[str, float] or None, optional
43         Radial bounds 'ir_cutoff' and 'uv_cutoff', with  $0 \leq IR \leq UV$ .
44         Required only under cutoff control; ignored under explicit-M control.
45     CTRL_M_EXPLICIT : bool, optional
46         True selects exactly M modes; False selects  $IR \leq |k_j| \leq UV$ .
47     M : int or None, optional
48         Positive odd mode count when explicit control is active. Its indices
49         run from  $-(M-1)/2$  to  $+(M-1)/2$  and include zero. Otherwise ignored.
50
51     Returns
52     -----
53     numpy.ndarray
54         Sorted float array of shape (M_base,), including both signed momenta.
55         Cutoff control includes zero exactly when IR is zero. The count follows
56         from the band, rather than from the inactive M argument.
57
58     Raises
59     -----
60     ValueError
61         Invalid length, bounds, control flag or explicit count, or an empty band.
62     """
63     L = float(param_atom["L"])
64     if not np.isfinite(L) or L <= 0:
65         raise ValueError("L must be finite and positive")
66     if not isinstance(CTRL_M_EXPLICIT, bool):
67         raise ValueError("CTRL_M_EXPLICIT must be a boolean")
68     spacing = 2 * np.pi / L
69     if CTRL_M_EXPLICIT:
70         M = integer(M, "M", minimum=1)
71         if M % 2 == 0:
72             raise ValueError("M must be odd for a symmetric grid containing k=0")
73         half = (M - 1) // 2
74         indices = np.arange(-half, half + 1)
75     else:
76         if cutoffs is None:
77             raise ValueError("IR/UV cutoffs are required when CTRL_M_EXPLICIT=False")
78         ir, uv = float(cutoffs["ir_cutoff"]), float(cutoffs["uv_cutoff"])
79         if not np.isfinite(ir) or not np.isfinite(uv) or not 0 <= ir <= uv:
80             raise ValueError("Cutoffs must be finite and satisfy  $0 \leq ir\_cutoff \leq uv\_cutoff$ ")
81         lower = int(np.ceil(ir / spacing - 1e-12))
82         upper = int(np.floor(uv / spacing + 1e-12))
83         positive = np.arange(max(1, lower), upper + 1)
84         zero = np.array([0], dtype=int) if ir == 0 else np.array([], dtype=int)
85         indices = np.concatenate((-positive[::-1], zero, positive))
86     modes = spacing * indices
87     if len(modes) == 0:
88         raise ValueError("The momentum band contains no modes")
89     return modes

```

```

90
91
92 def select_modes(k_tab, param_photon, param_atom, photon_window, atom_window):
93     """Select the union of packet and atomic-resonance momentum windows.
94
95     Parameters
96     -----
97     k_tab : numpy.ndarray
98         Nonempty float array of shape (M_base,), containing signed momenta.
99     param_photon : dict[str, float]
100         Packet centre 'k_0' and positive width 'sigma_k'.
101     param_atom : dict[str, float]
102         Atomic frequency 'omega_0'; resonance centres are +/-omega_0.
103     photon_window : float
104         Nonnegative factor w_p in |k-k_0| <= w_p*sigma_k.
105     atom_window : float
106         Nonnegative factor w_a for |k-omega_0| <= w_a*sigma_k or
107         |k+omega_0| <= w_a*sigma_k.
108
109     Returns
110     -----
111     numpy.ndarray
112         Boolean mask of shape (M_base,). Each empty window contributes its
113         nearest mode (first array entry in a tie), so the union is nonempty.
114         A 1e-12 absolute boundary tolerance is used. Ordering is preserved;
115         this optional grid restriction does not impose an incident direction.
116     """
117     sigma = float(param_photon["sigma_k"])
118     if not np.isfinite(sigma) or sigma <= 0:
119         raise ValueError("sigma_k must be finite and positive")
120     if any(not np.isfinite(w) or w < 0 for w in (photon_window, atom_window)):
121         raise ValueError("Selection windows must be finite and nonnegative")
122     centres = ((param_photon["k_0"], photon_window * sigma),
123               (param_atom["omega_0"], atom_window * sigma),
124               (-param_atom["omega_0"], atom_window * sigma))
125     mask = np.zeros(len(k_tab), dtype=bool)
126     for centre, radius in centres:
127         window = np.abs(k_tab - centre) <= radius + 1e-12
128         if not window.any():
129             window[np.argmin(np.abs(k_tab - centre))] = True
130         mask |= window
131     return mask
132
133
134 def resolve_grid(config):
135     """Resolve the base grid and the actual momenta used by a configuration.
136
137     Parameters
138     -----
139     config : ExperimentConfig
140         Carries L, grid-control inputs and optional packet/resonance windows.
141
142     Returns
143     -----
144     base, selected : tuple[numpy.ndarray, numpy.ndarray]
145         Sorted float arrays of shapes (M_base,) and (M_selected,). Selection
146         acts after base-grid construction. With selection disabled, selected
147         is a separate copy of base. Both engine and estimator use this routine.
148     """
149     base = momentum_modes(config.param_atom, config.cutoffs, config.CTRL_M_EXPLICIT, config.M)
150     if config.mode_selection:
151         mask = select_modes(base, config.param_photon, config.param_atom,
152                             config.photon_window, config.atom_window)
153         return base, base[mask]
154     return base, base.copy()
155
156
157 def grid_summary(base, selected, L):
158     """Describe effective cutoffs and exact equivalence of the two grid controls.
159
160     Parameters
161     -----
162     base : numpy.ndarray
163         Nonempty float array (M_base,) of periodic-box momenta.
164     selected : numpy.ndarray

```

```

165     Nonempty float array (M_selected,) retained after optional windows.
166 L : float
167     Positive box length; delta_k = 2*pi/L recovers integer mode indices.
168
169 Returns
170 -----
171 dict[str, object]
172     'delta_k' (float) and 'base'/'selected' dictionaries with mode
173     counts, signed extrema, effective IR=min|k|, UV=max|k|, smallest nonzero
174     |k| (or None), zero presence and integer indices. 'radial_cutoffs_exact'
175     tests the entire symmetric band, including holes. 'equivalent_M' is
176     an int only for a complete symmetric band containing zero; else None.
177 """
178 def describe(k_tab):
179     """Describe one grid using its integer periodic-box labels.
180
181     Parameters
182     -----
183     k_tab : numpy.ndarray
184         Nonempty float momentum array (M,); enclosing L sets delta_k.
185
186     Returns
187     -----
188     dict[str, object]
189         Count, extrema, zero policy, integer labels and exact representability.
190         Equality with {j: j_min <= |j| <= j_max} detects radial-band completeness.
191     """
192     indices = np rint(k_tab / (2 * np.pi / L)).astype(int)
193     lower, upper = int(np.abs(indices).min()), int(np.abs(indices).max())
194     expected = {n for n in range(-upper, upper + 1) if abs(n) >= lower}
195     radial_exact = set(indices) == expected
196     nonzero = np.abs(k_tab[k_tab != 0])
197     return {"n_modes": len(k_tab), "k_min": float(k_tab.min()),
198           "k_max": float(k_tab.max()), "ir_effective": float(np.abs(k_tab).min()),
199           "uv_effective": float(np.abs(k_tab).max()),
200           "smallest_nonzero_k": float(nonzero.min()) if len(nonzero) else None,
201           "zero_present": bool(np.any(k_tab == 0)),
202           "radial_cutoffs_exact": radial_exact,
203           "equivalent_M": len(k_tab) if lower == 0 and radial_exact else None,
204           "integer_indices": indices.tolist()}
205 return {"delta_k": 2 * np.pi / L, "base": describe(base), "selected": describe(selected)}

```

Lines	Mathematical and implementation commentary
1–34	integer accepts integral types above its minimum, including NumPy integers. It rejects booleans and all floats rather than silently coercing them.
35–62	One grid generator for both evolution and estimation; its inputs describe the base grid, before any optional windows.
63–67	Convert and require finite $L > 0$. Require an actual boolean control so a truthy string cannot silently select the wrong branch.
68–68	Compute the physical spacing $\Delta k = 2\pi/L$, (1); it is never inferred from a chosen number of modes.
69–75	Require positive odd integer M and construct indices $-J, \dots, J$, $J = (M-1)/2$. Even counts raise an error before basis allocation.
76–80	Cutoff control requires finite ordered radial bounds $0 \leq \text{IR} \leq \text{UV}$. These validation steps are skipped when M controls the grid.
81–82	Ceiling/floor of cutoff ratios with dimensionless 10^{-12} boundary tolerances. The positive lower index is subsequently restricted to at least one; compare (5).
83–85	Enumerate positive indices, reflect them in reversed order for the negative side, and add zero only when input IR is exactly zero. A tiny positive IR is not rounded to zero.
86–91	Multiply integer indices by Δk and reject an empty band. Sorted signed momenta, not frequencies, are returned.

Lines	Mathematical and implementation commentary
92–121	Validate positive finite σ_k and nonnegative finite factors. This is a requested subset operation; it is not a right-moving preparation filter.
122–124	Window centres $k_0, +\omega_0, -\omega_0$ with radii $w_p\sigma_k, w_a\sigma_k$, as in (7).
125–133	For each window use an absolute momentum tolerance, add the nearest mode if empty, and union its mask. Ties use the first sorted entry. The returned mask preserves base-grid ordering.
134–156	Resolve the same base grid for estimator and engine. With selection return its indexed subset; otherwise return a distinct array copy with the same values.
157–193	Recover integer indices from the physical spacing and compute radial extrema. This representation report is independent of the photon cap and basis type.
194–196	Test equality with the entire symmetric radial band, (65); minima/maxima alone would miss holes or unpaired endpoints. Compute the smallest nonzero $ k $ separately.
197–204	Report count, signed extrema, effective cutoffs, zero, integer indices and exact representability. An equivalent explicit M exists only for a radial-exact set containing zero.
205–205	Return separate base and selected descriptions with their common physical spacing. A selected count may differ from the explicit base-grid input.

src/states.py

```

1  """Fock bases and Gaussian preparations; TLS is the last binary index."""
2
3  from itertools import combinations_with_replacement, product
4  from math import factorial
5
6  import numpy as np
7  import qutip
8
9  from .grid import integer
10
11
12  class FockBasis:
13      """Occupation tuples and joint field/TLS indexing for one finite basis.
14
15      Attributes
16      -----
17      states : list[tuple[int, ...]]
18          B retained field occupations n=(n_0,...,n_{M-1}); each tuple has M entries.
19      index : dict[tuple[int, ...], int]
20          Inverse lookup n -> i. Joint ket component q=2*i+s uses s=0 (g), 1 (e).
21      photon_numbers : numpy.ndarray
22          Integer array (B,) with sum_m n_m for each occupation, excluding the TLS.
23      modes, cap : int
24          Retained mode count M and photon cutoff N.
25      truncation : str
26          Basis constraint used to enumerate states.
27      dim : int
28          Joint ket dimension d=2*B, including the TLS binary factor.
29      """
30      def __init__(self, modes, cap, truncation):
31          """Enumerate the retained field occupations and their TLS indexing.
32
33      Parameters
34      -----
35      modes : int
36          Positive number M of retained physical field modes.
37      cap : int
38          Nonnegative photon cutoff N ('n_max').
39      truncation : {'truncated', 'full+totalcap', 'full'}
40          Retain vacuum plus n*e_m; all tuples with sum(n_m)<=N; or all tuples
41          with each n_m<=N, respectively. Their field counts B are 1+M*N,
42          binomial(M+N,N), and (N+1)**M. e_m denotes the unit occupation vector.
43
44      Returns

```

```

45 -----
46 None
47     Sets states, index, photon_numbers and dim on this instance. Vacuum is
48     included for every scheme. Enumeration order is scheme-dependent; use
49     index rather than assuming a tuple occupies the same position elsewhere.
50     """
51     self.modes = modes = integer(modes, "modes", minimum=1)
52     self.cap = cap = integer(cap, "n_max")
53     self.truncation = truncation
54     vacuum = (0,) * modes
55     if truncation == "truncated":
56         field_states = [vacuum]
57         for m in range(modes):
58             for n in range(1, cap + 1):
59                 occupation = list(vacuum)
60                 occupation[m] = n
61                 field_states.append(tuple(occupation))
62     elif truncation == "full+totalcap":
63         field_states = []
64         for total in range(cap + 1):
65             for occupied in combinations_with_replacement(range(modes), total):
66                 field_states.append(tuple(np.bincount(occupied, minlength=modes)))
67     elif truncation == "full":
68         field_states = list(product(range(cap + 1), repeat=modes))
69     else:
70         raise ValueError(f"Unknown truncation: {truncation}")
71     self.states = field_states
72     self.index = {occupation: i for i, occupation in enumerate(field_states)}
73     self.photon_numbers = np.array([sum(n) for n in field_states])
74     self.dim = 2 * len(field_states)
75
76
77 def initial_state(basis, k_tab, param_photon, param_atom):
78     """Prepare and normalize a projected Gaussian field state times a TLS state.
79
80     Parameters
81     -----
82     basis : FockBasis
83         Retained occupation set, with d=2*B joint ket components.
84     k_tab : array_like of float
85         Shape (M,), one signed momentum per occupation entry in basis order.
86     param_photon : dict[str, object]
87         Finite 'k_0', 'x_0', positive 'sigma_k'; 'state' is 'number'
88         (default) or 'coherent'. Number preparation uses integer 'n' (default
89         1, 0<=n<=N); coherent preparation uses finite complex 'alpha' (default 1).
90     param_atom : dict[str, object]
91         Optional 'initial_state': 'g' (default), 'e', '+', or '-'; the latter
92         two are (|g> +/- |e>)/sqrt(2).
93
94     Returns
95     -----
96     qutip.Qobj
97         Normalized ket of shape (d,1), with components q=2*i+s (s=0:g, 1:e).
98         QuTiP stores flat dimensions; the occupation/TLS split is given by basis.
99
100     Notes
101     -----
102     Packet amplitudes are c_m proportional to
103     exp(-(k_m-k_0)**2/(4*sigma_k**2)) * exp(-i*k_m*x_0), with sum|c_m|**2=1.
104     Every supplied signed mode participates. Subtracting the largest Gaussian
105     exponent prevents common underflow without changing normalized amplitudes.
106     For n>0, the field state is sum_m c_m |n*e_m>; for n=0 it is vacuum once.
107     This n>1 convention differs from (sum_m c_m a_m^dagger)**n |0>/sqrt(n!).
108     For coherent input, each retained tuple receives
109     prod_m (alpha*c_m)**n_m/sqrt(n_m!), followed by whole-ket normalization.
110     The omitted common exp(-|alpha|**2/2) cancels in that projection normalization.
111     """
112     k0, sigma = float(param_photon["k_0"]), float(param_photon["sigma_k"])
113     x0 = float(param_photon["x_0"])
114     if not np.isfinite([k0, sigma, x0]).all() or sigma <= 0:
115         raise ValueError("k_0 and x_0 must be finite; sigma_k must be finite and positive")
116     k_tab = np.asarray(k_tab, dtype=float)
117     exponent = -(k_tab - k0) ** 2 / (4 * sigma ** 2)
118     packet = np.exp(exponent - exponent.max()) * np.exp(-1j * k_tab * x0)
119     packet /= np.linalg.norm(packet)

```

```

120 atom = param_atom.get("initial_state", "g")
121 atom_coeffs = {"g": (1, 0), "e": (0, 1),
122               "+": (1 / np.sqrt(2), 1 / np.sqrt(2)),
123               "-": (1 / np.sqrt(2), -1 / np.sqrt(2))}
124 if atom not in atom_coeffs:
125     raise ValueError("initial_state must be g, e, +, or -")
126 atom_vector = np.asarray(atom_coeffs[atom], dtype=complex)
127 vector = np.zeros(basis.dim, dtype=complex)
128 kind = param_photon.get("state", "number")
129 if kind == "number":
130     number = integer(param_photon.get("n", 1), "n")
131     if number > basis.cap:
132         raise ValueError("n must satisfy 0 <= n <= n_max")
133     if number == 0:
134         i = basis.index[(0,) * basis.modes]
135         vector[2 * i:2 * i + 2] = atom_vector
136     else:
137         for m, amplitude in enumerate(packet):
138             occupation = [0] * basis.modes
139             occupation[m] = number
140             i = basis.index[tuple(occupation)]
141             vector[2 * i:2 * i + 2] = amplitude * atom_vector
142 elif kind == "coherent":
143     alpha = complex(param_photon.get("alpha", 1.0))
144     if not np.isfinite(alpha):
145         raise ValueError("alpha must be finite")
146     for i, occupation in enumerate(basis.states):
147         coefficient = 1.0 + 0j
148         for m, n in enumerate(occupation):
149             coefficient *= (alpha * packet[m]) ** n / np.sqrt(float(factorial(n)))
150         vector[2 * i:2 * i + 2] = coefficient * atom_vector
151 else:
152     raise ValueError("state must be number or coherent")
153 norm = np.linalg.norm(vector)
154 if not np.isfinite(norm) or norm == 0:
155     raise ValueError("The projected initial state cannot be normalized")
156 return qutip.Qobj(vector / norm)

```

Lines	Mathematical and implementation commentary
1–11	Combinations with replacement enumerate total-cap occupations; a Cartesian product enumerates full. These algorithms avoid testing every candidate product against a total cap.
12–53	Validate at least one retained mode and nonnegative integer cap. Store the representation name; $N = 0$ is a valid vacuum-only photon space.
54–54	One zero tuple of length M represents the photon vacuum. TLS components are attached later through the alternating index rule (14).
55–61	Append ne_m in mode-first, occupation-second order after vacuum. Multimode configurations are absent at every cap; dimension $2(1 + MN)$.
62–66	For each total from zero to N , enumerate multisets of occupied mode indices. <code>bincount</code> produces the tuple without duplicates; dimension $2^{\binom{M+N}{N}}$.
67–70	Cartesian product of $0, \dots, N$ on each mode. The final occupation changes fastest. Unknown names, including the removed extra-oscillator scheme, raise an error.
71–76	Store the exact tuples, their dictionary indices and photon totals. Ket dimension is twice the configuration count because the TLS remains a separate binary factor.
77–116	Read finite signed k_0 , finite x_0 and positive finite σ_k . All values in the supplied selected momentum array enter preparation.
117–119	Gaussian amplitude and phase from (15). Subtract the largest real exponent, then normalize the whole packet; the common stabilizing factor cancels.
120–125	Define the normalized $g, e, +, -$ vectors in TLS order (g, e) and reject unknown initial labels. Ground is the default.

Lines	Mathematical and implementation commentary
126–128	Allocate the joint complex ket and select number or coherent preparation. The default is number, with $n = 1$ below.
129–136	Require integer $0 \leq n \leq N$. For $n = 0$ write the TLS vector onto vacuum exactly once, independent of how many modes exist.
137–141	For $n > 0$, place $c_m a_0\rangle$ at occupation ne_m in alternating components. This is (18), not the collective-mode Fock state (19).
142–145	Read and validate complex α . The algorithm projects a product of coherent states, not a single coherent state restricted to one ray.
146–152	Multiply $(\alpha c_m)^{n_m} / \sqrt{n_m!}$ over all modes for each allowed tuple, then multiply both TLS components. The common coherent exponential is omitted because it cancels in (22).
153–156	Normalize the complete projected ket once, rejecting zero or nonfinite norm. The retained coherent mass and its basis dependence are derived in (25); no intersection normalization is involved.

src/hamiltonians.py

```

1  """Projected H = H0 + D V, with u_m = i f_m exp(-i k_m x_tls) / sqrt(L)."""
2
3  import numpy as np
4  import qutip
5  from scipy.sparse import coo_matrix, diags
6
7
8  class Hamiltonian:
9      """Sparse projected Hamiltonian H(D)=H0+D*V in natural units hbar=c=1.
10
11      Attributes
12      -----
13      basis : FockBasis
14          Occupation tuples and alternating ground/excited joint indices.
15      k_tab : numpy.ndarray
16          Signed momenta (M,); field frequencies are |k_m|.
17      param_atom : dict[str, object]
18          Physical parameters retained by reference, including D and coupling.
19      RWA : bool
20          Whether only excitation-conserving interaction terms are retained.
21      H0, V : scipy.sparse.csr_matrix
22          Operators of shape (d,d), d=basis.dim. V excludes the physical scale D.
23      """
24      def __init__(self, basis, k_tab, param_atom, RWA=False):
25          """Construct the free and interaction matrices on the supplied finite basis.
26
27          Parameters
28          -----
29          basis : FockBasis
30              Retained occupations; q=2*i+s combines field index i with TLS label s.
31          k_tab : array_like of float
32              Shape (basis.modes,), signed physical momenta in occupation order.
33          param_atom : dict[str, object]
34              'L': finite positive box length; 'omega_0': nonnegative TLS
35              excitation energy; 'x_tls': finite TLS coordinate in the phase;
36              'coupling': 'sqrt'/'flat' form factor; 'D': real physical
37              interaction scale used when building H=H0+D*V.
38          RWA : bool, optional
39              True keeps a_m*sigma_+ and a_m^dagger*sigma_- only.
40
41          Returns
42          -----
43          None
44              Sets CSR matrices H0 and V. This eagerly allocates sparse operators;
45              resource estimation should precede construction for large bases.
46          """
47          self.basis = basis
48          self.k_tab = np.asarray(k_tab)
49          self.param_atom = param_atom
50          self.RWA = RWA

```

```

51     L, omega, x = (float(param_atom[key]) for key in ("L", "omega_0", "x_tls"))
52     if not np.isfinite([L, omega, x]).all() or L <= 0 or omega < 0:
53         raise ValueError("L must be positive, omega_0 nonnegative, and x_tls finite")
54     self.H0 = self.free()
55     self.V = self.interaction()
56
57 def free(self):
58     """Build the diagonal projected free operator with atomic ground energy zero.
59
60     Parameters
61     -----
62     None
63
64     Uses this instance's basis, k_tab and param_atom['omega_0'].
65
66     Returns
67     -----
68     scipy.sparse.csr_matrix
69         Shape (d,d); diagonal entry at q=2*i+s is
70         E_(i,s) = sum_m |k_m|*n_(i,m) + s*omega_0, with s=0:g and s=1:e.
71         No zero-point field energy or atomic -omega_0/2 offset is added.
72     """
73     energies = np.zeros(self.basis.dim)
74     for i, occupation in enumerate(self.basis.states):
75         field_energy = np.dot(np.abs(self.k_tab), occupation)
76         energies[2 * i] = field_energy
77         energies[2 * i + 1] = field_energy + self.param_atom["omega_0"]
78     return diags(energies, format="csr")
79
80 def interaction(self):
81     """Assemble the projected interaction V, including the imposed complex phase.
82
83     Parameters
84     -----
85     None
86
87     Uses basis, signed k_tab, L, x_tls, coupling profile and RWA.
88
89     Returns
90     -----
91     scipy.sparse.csr_matrix
92         Shape (d,d), excluding D. Without RWA,
93         V=P[sum_m (u_m*a_m + u_m.conj()*a_m^dagger)*sigma_x]P,
94         where u_m=i*f_m*exp(-i*k_m*x_tls)/sqrt(L), and f_m=sqrt(|k_m|)
95         for 'sqrt', or 1 for 'flat'. P projects onto the chosen joint basis.
96
97     Notes
98     -----
99     Each source column q=2*i+s and target row 2*j+(1-s) differ by one photon.
100     Annihilation contributes u_m*sqrt(n_m); creation contributes
101     u_m.conj()*sqrt(n_m+1). Out-of-basis targets are omitted. Under RWA,
102     annihilation starts from g and creation from e, preserving sum(n_m)+s.
103     COO row/column/value buffers are temporary; the returned operator is CSR.
104     """
105     profile = self.param_atom["coupling"]
106     if profile == "sqrt":
107         form_factor = np.sqrt(np.abs(self.k_tab))
108     elif profile == "flat":
109         form_factor = np.ones(self.basis.modes)
110     else:
111         raise ValueError("coupling must be flat or sqrt")
112     u = 1j * form_factor * np.exp(-1j * self.k_tab * self.param_atom["x_tls"])
113     u /= np.sqrt(self.param_atom["L"])
114     rows, cols, values = [], [], []
115     for i, occupation in enumerate(self.basis.states):
116         for atom in (0, 1):
117             col = 2 * i + atom
118             for m in range(self.basis.modes):
119                 if occupation[m] and (not self.RWA or atom == 0):
120                     target = list(occupation)
121                     target[m] -= 1
122                     j = self.basis.index[tuple(target)]
123                     rows.append(2 * j + 1 - atom)
124                     cols.append(col)
125                     values.append(u[m] * np.sqrt(occupation[m]))
126                 if not self.RWA or atom == 1:
127                     target = list(occupation)

```

```

126         target[m] += 1
127         j = self.basis.index.get(tuple(target))
128         if j is not None:
129             rows.append(2 * j + 1 - atom)
130             cols.append(col)
131             values.append(u[m].conjugate() * np.sqrt(occupation[m] + 1))
132     return coo_matrix((values, (rows, cols)), shape=(self.basis.dim, self.basis.dim)).tocsr()
133
134 def build_hamiltonian(self, D=None):
135     """Combine the projected operators into H=H0+D*V for evolution.
136
137     Parameters
138     -----
139     D : float or None, optional
140         Finite real coupling scale. None uses param_atom['D']. This symbol is
141         the physical D convention; it is not the student's historical g.
142
143     Returns
144     -----
145     qutip.Qobj
146         Sparse operator of shape (d,d) with flat QuTiP dimensions. Does not
147         modify H0, V or the stored parameter dictionary. Real D ensures
148         Hermiticity of the paired projected transitions.
149     """
150     D = self.param_atom["D"] if D is None else D
151     if np.iscomplexobj(D) or not np.isfinite(D):
152         raise ValueError("D must be finite and real")
153     return qutip.Qobj(self.H0 + float(D) * self.V)

```

Lines	Mathematical and implementation commentary
1–7	Use COO triplets and CSR storage rather than a dense $d \times d$ allocation. QuTiP wraps the final sparse operator for evolution.
8–56	Store basis and physical parameters; require finite $L > 0$, $\omega_0 \geq 0$ and finite TLS position. Build the free term and interaction separately.
57–78	For each occupation compute $\sum_m k_m n_m$, then place it on ground and add ω_0 on excited TLS. The ground-state energy origin is explicit, (33).
79–109	Select $f_m = \sqrt{ k_m }$ or 1; reject other labels. The signed momentum survives in the spatial phase even though the profile uses $ k $.
110–111	Compute $u_m = i f_m e^{-i k_m x_{\text{tls}}} / \sqrt{L}$, (30). D is not multiplied yet, allowing the same interaction matrix to be scaled.
112–112	Temporary row/column/value lists hold the sparse interaction entries. A column labels the joint source ket and a row its target; coefficients contain u_m but not D .
113–116	Iterate occupations, TLS labels and changed modes. The matrix column is the source $2i + s$; the target row flips the TLS to $1 - s$.
117–122	Annihilation requires positive occupation. Under RWA it also requires source TLS g . All retained bases are downward closed, so dictionary lookup of the decremented tuple is valid.
123–123	Record $u_m \sqrt{n_m}$. The factor is the bosonic matrix element, with the imposed complex annihilation phase.
124–127	Creation is allowed for either TLS without RWA and only source e under RWA. The incremented tuple is tested, not assumed present.
128–131	Omit targets outside the chosen space, exactly implementing projection. Retained creation receives $u_m^* \sqrt{n_m + 1}$, conjugate to its reverse annihilation.
132–133	Convert temporary COO row/column/value records to the CSR interaction matrix. The assembly buffers are not kept as separate diagnostic arrays; zero coefficients can remain as stored sparse entries.

Lines	Mathematical and implementation commentary
134–153	Require finite real D and return $H_0 + DV$ as a QuTiP operator. Hermiticity follows from paired transitions and real scaling, not solver output normalization.

src/fidelities.py

```

1 """Exact squared state/TLS fidelities in an implicit common full basis."""
2
3 import numpy as np
4 import qutip
5
6
7 def _vector(ket, basis):
8     """Validate and normalize a complete joint ket for fidelity evaluation.
9
10     Parameters
11     -----
12     ket : qutip.Qobj or numpy.ndarray
13         Ket (d,1) as Qobj, or a one-dimensional array (d,).
14     basis : FockBasis
15         Expected joint dimension d and occupation/TLS ordering.
16
17     Returns
18     -----
19     numpy.ndarray
20         Normalized ket (d,), v/||v||, preserving every component. Zero or
21         nonfinite norm and incompatible dimensions raise ValueError. This
22         normalization does not modify the original solver state.
23     """
24     vector = ket.full()[:, 0] if isinstance(ket, qutip.Qobj) else np.asarray(ket)
25     if vector.shape != (basis.dim,):
26         raise ValueError("Ket dimension does not match the Fock basis")
27     norm = np.linalg.norm(vector)
28     if not np.isfinite(norm) or norm == 0:
29         raise ValueError("A fidelity requires nonzero finite kets")
30     return vector / norm
31
32
33 def atom_density_matrix(vector):
34     """Trace out the field using the alternating TLS component convention.
35
36     Parameters
37     -----
38     vector : array_like of complex
39         Joint vector (2*B,), v[2*i+s]=C_(i,s), s=0:g and 1:e.
40
41     Returns
42     -----
43     numpy.ndarray
44         Complex matrix (2,2) in (g,e) order:
45         rho_(s,s') = sum_i C_(i,s)*C_(i,s').conj() = (C.T @ C.conj())_(s,s').
46         The trace is ||v||**2. Input is not normalized by this helper; divide
47         by the squared ket norm when a unit-trace density matrix is needed.
48     """
49     coefficients = np.asarray(vector).reshape(-1, 2)
50     return coefficients.T @ coefficients.conj()
51
52
53 def _mode_maps(k_a, k_b):
54     """Map two physical momentum lists into one implicit union of field modes.
55
56     Parameters
57     -----
58     k_a, k_b : array_like of float
59         Shapes (M_a,) and (M_b,). Each entry denotes a distinct physical mode;
60         callers must ensure compatible box lengths and physical conventions.
61
62     Returns
63     -----
64     map_a, map_b : tuple[list[int], list[int]]
65         Local mode index -> union index maps, of lengths M_a and M_b. Union
66         starts in k_a order; unseen k_b entries are appended. Matching uses

```

```

67         absolute tolerance 1e-12 and no relative tolerance. Multiple possible
68         matches raise ValueError rather than identifying an ambiguous mode.
69     """
70     union = list(np.asarray(k_a, dtype=float))
71     map_a = list(range(len(union)))
72     map_b = []
73     for k in k_b:
74         matches = np.flatnonzero(np.isclose(union, k, atol=1e-12, rtol=0))
75         if len(matches) > 1:
76             raise ValueError("Ambiguous physical momentum alignment")
77         if len(matches):
78             map_b.append(int(matches[0]))
79         else:
80             map_b.append(len(union))
81             union.append(float(k))
82     return map_a, map_b
83
84
85 def _components(basis, vector, mapping):
86     """Represent each physical occupation by its two TLS amplitudes.
87
88     Parameters
89     -----
90     basis : FockBasis
91         Source occupations, each of length M.
92     vector : numpy.ndarray
93         Complex joint ket (basis.dim,) in alternating (g,e) order.
94     mapping : list[int]
95         Length M; maps local modes to indices of the common physical union.
96
97     Returns
98     -----
99     dict[tuple[tuple[int, int], ...], numpy.ndarray]
100         Canonical key -> TLS pair (2,). A key is the sorted tuple of
101         (union_mode_index, positive_occupation) pairs; vacuum has key ().
102         Omitted modes therefore carry zero occupation. Values are array views,
103         and multimode occupations are retained without an ambient full vector.
104     """
105     amplitudes = {}
106     for i, occupation in enumerate(basis.states):
107         key = tuple(sorted((mapping[m], int(n)) for m, n in enumerate(occupation) if n))
108         amplitudes[key] = vector[2 * i : 2 * i + 2]
109     return amplitudes
110
111
112 def compare_states(ket_a, basis_a, k_a, ket_b, basis_b, k_b):
113     """Compute squared state and TLS fidelities in a common full occupation space.
114
115     Parameters
116     -----
117     ket_a, ket_b : qutip.Qobj or numpy.ndarray
118         Joint kets (d_a,1)/(d_b,1) as Qobj or arrays (d_a,)/(d_b,).
119     basis_a, basis_b : FockBasis
120         Their occupation sets and alternating TLS indexing; caps may differ.
121     k_a, k_b : array_like of float
122         Physical mode lists (M_a,)/(M_b,) matching the respective tuple entries.
123         Caller guarantees identical box length and Hamiltonian conventions.
124
125     Returns
126     -----
127     dict[str, float]
128         'F_state' = |<I_a psi_a | I_b psi_b>|**2 and
129         'F_atom' = (Tr sqrt(sqrt(rho_a)*rho_b*sqrt(rho_a)))*2, clipped to [0,1]
130         against roundoff. psi_a/b are individually normalized whole kets;
131         rho_a/b are their field traces. QuTiP's root fidelity is squared here.
132
133     Notes
134     -----
135     I_a/b pad missing physical modes with vacuum in the union full basis.
136     The overlap sums TLS dot products over every shared occupation key. There
137     is no normalization on that intersection, and no full-space allocation.
138     The ambient full basis defines overlap only; it is not a simulated reference.
139     """
140     a, b = _vector(ket_a, basis_a), _vector(ket_b, basis_b)
141     map_a, map_b = _mode_maps(k_a, k_b)

```

```

142 components_a = _components(basis_a, a, map_a)
143 components_b = _components(basis_b, b, map_b)
144 overlap = sum(np.vdot(components_a[key], components_b[key])
145               for key in components_a.keys() & components_b.keys())
146 rho_a, rho_b = qutip.Qobj(atom_density_matrix(a)), qutip.Qobj(atom_density_matrix(b))
147 return {"F_state": float(np.clip(abs(overlap) ** 2, 0, 1)),
148        "F_atom": float(np.clip(qutip.fidelity(rho_a, rho_b) ** 2, 0, 1))}
149
150
151 def fidelities_over_time(experiment_a, experiment_b):
152     """Compare synchronized stored trajectories using the implicit full embedding.
153
154     Parameters
155     -----
156     experiment_a, experiment_b : Experiment
157         Propagated objects with store_state=True, equal output-time arrays and
158         box lengths equal to absolute tolerance 1e-12. Bases/mode subsets may
159         differ; physical mode identity and TLS conventions must agree.
160
161     Returns
162     -----
163     dict[str, numpy.ndarray]
164         'F_state' and 'F_atom' float arrays (N_t,), evaluated on every
165         corresponding pair of stored kets. Both are squared fidelities.
166
167     Raises
168     -----
169     ValueError
170         Either history is absent, times differ, or box lengths are incompatible.
171
172     """
173     if not experiment_a.store_state or not experiment_b.store_state:
174         raise ValueError("Both experiments must store states for a time series")
175     if not np.array_equal(experiment_a.times, experiment_b.times):
176         raise ValueError("Experiments must use the same output times")
177     if not np.isclose(experiment_a.param_atom["L"], experiment_b.param_atom["L"],
178                      atol=1e-12, rtol=0):
179         raise ValueError("Experiments must use the same box length for physical-mode embedding")
180     pairs = zip(experiment_a.result.states, experiment_b.result.states)
181     values = [compare_states(a, experiment_a.basis, experiment_a.k_tab,
182                             b, experiment_b.basis, experiment_b.k_tab) for a, b in pairs]
183     return {name: np.array([value[name] for value in values])
184            for name in ("F_state", "F_atom")}

```

Lines	Mathematical and implementation commentary
1–6	This module implements exactly two squared metrics; ambient full-space vectors and a ratio field proxy are not constructed.
7–32	Accept a QuTiP ket or one-dimensional array of the basis dimension. Normalize the entire nonzero finite ket for metric evaluation, retaining raw norm diagnostics elsewhere.
33–52	Reshape alternating components to C with shape $(B, 2)$, then compute $C^T C^*$ as in (40). For a raw ket its trace is the squared norm.
53–72	Start the union with the first physical momentum list and its identity map. Internal union order need not be sorted because occupation keys are canonicalized.
73–84	Match each second-grid mode with absolute tolerance 10^{-12} and zero relative tolerance. Reuse its union index, append if absent, or reject an ambiguous match.
85–111	For each tuple, discard zero occupations and sort positive (union index, n) pairs. Store the two TLS amplitudes; missing modes are vacuum implicitly and vacuum has an empty key.
112–143	Normalize both kets, align physical modes, and build occupation dictionaries. This low-level API cannot check box length or Hamiltonian conventions, which remain caller responsibilities.

Lines	Mathematical and implementation commentary
144–145	Sum the conjugate first-state times second-state TLS dot product over every shared physical key, (55). Do not normalize on that intersection; example (56) would otherwise be wrong.
146–150	Reduce the individually normalized kets, square QuTiP root fidelity, and return the two metrics clipped for numerical overshoot. Vacuum embedding leaves these atomic reductions unchanged.
151–178	Require both histories, exactly matching output arrays and equal L to absolute 10^{-12} . These checks precede pairing, preventing silent comparison of different time samples or boxes.
179–183	Compare matching stored states and return two one-dimensional arrays of length N_t . The simulated comparator is an independent input, not the ambient embedding choice.

src/experiment.py

```

1 """Pure-state dynamics and diagnostics shared by every experiment."""
2
3 from math import comb
4
5 import numpy as np
6 import qutip
7
8 from .fidelities import atom_density_matrix
9 from .grid import grid_summary, integer, momentum_modes, resolve_grid
10 from .hamiltonians import Hamiltonian
11 from .reporting import display_resource_tables, observables_table, resource_tables
12 from .states import FockBasis, initial_state
13 from .xp_config import ExperimentConfig
14
15
16 def output_times(param_time_evol):
17     """Construct requested output times including zero and the exact final time.
18
19     Parameters
20     -----
21     param_time_evol : dict[str, float]
22         Positive finite 'T' and 'dt'. dt controls output spacing, not the
23         solver's adaptive internal steps.
24
25     Returns
26     -----
27     numpy.ndarray
28         Float array (Nt), starting at 0 and ending at T. Interior samples
29         follow j*dt; the last interval may be shorter. If dt>T, returns [0,T].
30         Nonpositive/nonfinite inputs raise ValueError.
31     """
32     T, dt = float(param_time_evol["T"]), float(param_time_evol["dt"])
33     if not np.isfinite([T, dt]).all() or T <= 0 or dt <= 0:
34         raise ValueError("T and dt must be finite and positive")
35     count = int(np.floor(T / dt + 1e-12))
36     if count == 0:
37         return np.array([0.0, T])
38     times = np.arange(count + 1) * dt
39     if times[-1] < T and not np.isclose(times[-1], T, atol=1e-12, rtol=0):
40         times = np.append(times, T)
41     else:
42         times[-1] = T
43     return times
44
45
46 def resource_estimation(param_atom, param_time_evol, cutoffs=None,
47                         n_max=3, truncation="full+totalcap", store_state=True,
48                         CTRL_M_EXPLICIT=False, M=None, param_photon=None,
49                         mode_selection=False, photon_window=1.5, atom_window=0.25,
50                         print_report=True):
51     """Estimate basis dimensions and retained-vector memory before allocation.
52
53     Parameters

```

```

54 -----
55 param_atom : dict[str, object]
56     Contains L (positive float); omega_0 is also used for selected windows.
57 param_time_evol : dict[str, float]
58     T and dt determine the N_t requested outputs.
59 cutoffs : dict[str, float] or None, optional
60     ir_cutoff/uv_cutoff radial bounds when explicit-M control is disabled.
61 n_max : int, optional
62     Nonnegative photon cap N, default 3.
63 truncation : {'full+totalcap', 'truncated', 'full'}, optional
64     Joint dimensions  $d=2*\text{binomial}(M+N,N)$ ,  $2*(1+M*N)$ , or  $2*(N+1)**M$ .
65 store_state : bool, optional
66     True estimates N_t retained kets; False estimates one final ket.
67 CTRL_M_EXPLICIT : bool, optional
68     True chooses an odd positive M; False uses the cutoff band.
69 M : int or None, optional
70     Explicit base-grid mode count; ignored under cutoff control.
71 param_photon : dict[str, object] or None, optional
72     k_0 and sigma_k are required only when mode_selection is enabled.
73 mode_selection : bool, optional
74     Apply packet/resonance windows before computing the dimension.
75 photon_window, atom_window : float, optional
76     Window radii as multiples of sigma_k, default 1.5 and 0.25.
77 print_report : bool, optional
78     Display resource DataFrames in IPython or terminal tables. False
79     suppresses display but still returns the frames and numerical values.
80
81 Returns
82 -----
83 dict[str, object]
84     Mode count, d, N_t, memory estimates in GiB, sparse-entry upper bound,
85     heuristic feasible flag, grid descriptions, and pandas DataFrames
86     'grid_table'/'resource_table'. Existing scalar/dictionary keys remain
87     available. Grid reports show effective IR/UV and equivalent explicit M.
88
89 Notes
90 -----
91 One complex128 ket costs 16*d bytes. history_gib counts retained solver
92 kets; retained_vectors_gib approximates twice that for copied coefficients.
93 The upper bound nnz <= d*(1+2*M) includes the diagonal. Feasibility means
94 d<=100000 and retained_vectors_gib<=1; it is a heuristic, not a guarantee.
95 Sparse matrices, basis/Python objects, solver workspace and plots are
96 excluded (also recorded in resource_table.attrs). No basis is enumerated.
97 """
98 config = ExperimentConfig(param_photon or {}, param_atom, param_time_evol, cutoffs,
99                           n_max, truncation, store_state=store_state,
100                           CTRL_M_EXPLICIT=CTRL_M_EXPLICIT, M=M,
101                           mode_selection=mode_selection, photon_window=photon_window,
102                           atom_window=atom_window)
103 base, selected = resolve_grid(config)
104 modes, cap = len(selected), integer(n_max, "n_max")
105 if truncation == "full+totalcap":
106     dimension = 2 * comb(modes + cap, cap)
107 elif truncation == "truncated":
108     dimension = 2 * (1 + modes * cap)
109 elif truncation == "full":
110     dimension = 2 * (cap + 1) ** modes
111 else:
112     raise ValueError(f"Unknown truncation: {truncation}")
113 times = output_times(param_time_evol)
114 ket_gib = dimension * 16 / 2**30
115 history_gib = ket_gib * (len(times) if store_state else 1)
116 retained_gib = 2 * history_gib
117 nnz_bound = dimension * (1 + 2 * modes)
118 summary = grid_summary(base, selected, param_atom["L"])
119 feasible = dimension <= 100_000 and retained_gib <= 1.0
120 estimate = {"n_modes": modes, "dimension": dimension, "n_times": len(times),
121            "ket_gib": ket_gib, "history_gib": history_gib,
122            "retained_vectors_gib": retained_gib, "hamiltonian_nnz_bound": nnz_bound,
123            "feasible": feasible, "grid": summary}
124 estimate.update(resource_tables(estimate, config))
125 if print_report:
126     display_resource_tables(estimate)
127 return estimate
128

```

```

129
130 def estimate_config(config, print_report=True):
131     """Estimate resources using the exact inputs of a shared configuration.
132
133     Parameters
134     -----
135     config : ExperimentConfig
136         Grid, selection, photon cap, basis, output times and storage policy.
137     print_report : bool, optional
138         Display tabular estimates when True; False returns them silently.
139
140     Returns
141     -----
142     dict[str, object]
143         resource_estimation output, including numerical entries and reusable
144         'grid_table'/'resource_table' pandas DataFrames. No propagation occurs.
145     """
146     return resource_estimation(config.param_atom, config.param_timeevol, config.cutoffs,
147                               config.n_max, config.truncation, config.store_state,
148                               config.CTRL_M_EXPLICIT, config.M, config.param_photon,
149                               config.mode_selection, config.photon_window, config.atom_window,
150                               print_report)
151
152
153 class Experiment:
154     """One finite-basis evolution and its reusable physical diagnostics.
155
156     Attributes
157     -----
158     config : ExperimentConfig
159         Original configuration; physical dictionaries below are copied.
160     base_k_tab, k_tab : numpy.ndarray
161         Base and retained signed momenta ( $M_{\text{base}}$ )/( $M$ ).  $n_{\text{modes}}$  equals  $M$ .
162     times : numpy.ndarray
163         Requested physical output times ( $N_t$ ), including zero and  $T$ .
164     basis : FockBasis
165         B field tuples and  $d=2*B$  joint ket components  $q=2*i+s$ ,  $s=0:g/1:e$ .
166     hamiltonian : Hamiltonian
167         Sparse operators  $H_0$  and  $V$ ;  $H$  is the corresponding QuTiP  $H_0+D*V$ .
168     state0 : qutip.Qobj
169         Normalized projected initial ket ( $d,1$ ).
170     result : qutip.solver.Result
171         Set by propagate_state; stored kets and the final state from sesolve.
172     c_g_array, c_e_array : numpy.ndarray or None
173         Complex amplitudes  $C_{(i,g/e)}(t)$ . Shape ( $N_t, B$ ) with history or ( $B$ ,)
174         for final-only storage. Initially None; slices share coefficient memory.
175     observables : dict[str, numpy.ndarray] or None
176         Raw population diagnostics, initially None, then arrays ( $K$ ,) with
177          $K=N_t$  for histories or  $K=1$  for final-only storage.
178     """
179     def __init__(self, config):
180         """Build one grid, basis, Hamiltonian and normalized initial ket.
181
182         Parameters
183         -----
184         config : ExperimentConfig
185             Complete physical and numerical inputs. Dictionary contents are copied
186             to the instance; the original configuration itself is retained.
187
188         Returns
189         -----
190         None
191
192         Initializes the objects documented on Experiment. Allocates the basis,
193         sparse matrices and initial ket immediately, but does not propagate.
194         Call estimate_config first when allocation may be large.
195     """
196     self.config = config
197     self.param_photon, self.param_atom = dict(config.param_photon), dict(config.param_atom)
198     self.param_timeevol = dict(config.param_timeevol)
199     self.n_max, self.truncation = config.n_max, config.truncation
200     self.RWA, self.store_state = config.RWA, config.store_state
201     self.base_k_tab, self.k_tab = resolve_grid(config)
202     self.n_modes = len(self.k_tab)
203     self.times = output_times(self.param_timeevol)
204     self.basis = FockBasis(self.n_modes, self.n_max, self.truncation)

```

```

204 self.photon_numbers = self.basis.photon_numbers
205 self.hamiltonian = Hamiltonian(self.basis, self.k_tab, self.param_atom, self.RWA)
206 self.H = self.hamiltonian.build_hamiltonian()
207 self.state0 = initial_state(self.basis, self.k_tab, self.param_photon, self.param_atom)
208 self.c_g_array = self.c_e_array = self.observables = None
209
210 def propagate_state(self, progress=False):
211     """Solve  $i\hbar d|\psi\rangle/dt = H|\psi\rangle$  and retain raw field/TLS amplitudes.
212
213     Parameters
214     -----
215     progress : bool, optional
216         Enable the QuTiP tqdm progress bar when True.
217
218     Returns
219     -----
220     c_g_array, c_e_array : tuple[numpy.ndarray, numpy.ndarray]
221         Complex arrays ( $N_t, B$ ) for histories, or ( $B,$ ) for final-only storage.
222         Entry  $i$  is the amplitude of field occupation basis.states[ $i$ ] with TLS
223          $g$  or  $e$ . Also stores self.result and these arrays on the instance.
224
225     Notes
226     -----
227     Default solver is BDF with  $rtol=1e-9$ ,  $atol=1e-11$ , overridable through
228     param_time_evol.normalize_output=False retains numerical norm drift.
229     The final ket is always stored. c_g/c_e are alternating views of a copied
230     joint coefficient array, in addition to QuTiP's retained state objects.
231     """
232     self.result = qutip.sesolve(self.H, self.state0, self.times, options={
233         "method": self.param_time_evol.get("method", "bdf"),
234         "store_states": self.store_state, "store_final_state": True,
235         "normalize_output": False, "rtol": self.param_time_evol.get("rtol", 1e-9),
236         "atol": self.param_time_evol.get("atol", 1e-11),
237         "progress_bar": "tqdm" if progress else False})
238     vectors = np.array([s.full()[0] for s in self.result.states]) if self.store_state \
239         else self.result.final_state.full()[0]
240     self.c_g_array, self.c_e_array = vectors[..., 0::2], vectors[..., 1::2]
241     return self.c_g_array, self.c_e_array
242
243 def _one_photon_indices(self):
244     """Locate one-photon occupations and map them to physical field modes.
245
246     Parameters
247     -----
248     None
249     Uses basis.states and the corresponding total photon_numbers.
250
251     Returns
252     -----
253     indices, mode_ids : tuple[numpy.ndarray, numpy.ndarray]
254         Integer arrays ( $J,$ ),  $J=M$  when  $n_{\max} \geq 1$  and zero otherwise. indices
255         are field-occupation indices  $i$  (not joint indices  $q$ ); mode_ids gives
256         the sole occupied mode  $m$  of each tuple  $n=e_m$ . No basis order is assumed.
257     """
258     indices = np.flatnonzero(self.photon_numbers == 1)
259     mode_ids = np.array([np.argmax(self.basis.states[i]) for i in indices], dtype=int)
260     return indices, mode_ids
261
262 def compute_observables(self):
263     """Compute raw norm, TLS excitation and field-sector populations after evolution.
264
265     Parameters
266     -----
267     None
268     Uses propagated c_g/c_e arrays and the retained occupation set.
269
270     Returns
271     -----
272     dict[str, numpy.ndarray]
273         Real arrays ( $K,$ ),  $K=N_t$  or 1 for final-only. Includes time,
274          $\text{norm}=\sum_i |C(i,s)|^2$ ,  $p_{\text{excited}}=\sum_i |C(i,e)|^2$ , and
275          $p_{n\text{-photon}}=\sum_i (\sum_n n_i) |C(i,s)|^2$  for every retained  $n$ .
276          $p_{\text{transmitted}}/p_{\text{reflected}}/p_{\text{zero-1g}}$  sum  $|C(e_m,g)|^2$  over  $k_m > 0$ ,
277          $k_m < 0$ , or  $k_m = 0$ . These directional entries use only the one-photon,
278         ground-TLS sector. Stores the dictionary as self.observables.

```

```

279
280 Notes
281 -----
282 Requires propagate_state. No norm division is applied: sum_n p_n_photon
283 equals norm, not necessarily exactly one. Directional populations are
284 raw channel occupations, not automatically asymptotic scattering ratios.
285 """
286 if self.c_g_array is None:
287     raise RuntimeError("Call propagate_state first")
288 ground, excited = np.atleast_2d(self.c_g_array), np.atleast_2d(self.c_e_array)
289 probability = np.abs(ground) ** 2 + np.abs(excited) ** 2
290 indices, modes = self._one_photon_indices()
291 self.observables = {"time": self.times if self.store_state else self.times[-1:],
292                     "norm": probability.sum(axis=1),
293                     "p_excited": (np.abs(excited) ** 2).sum(axis=1)}
294 for name, mask in (("p_transmitted", self.k_tab[modes] > 0),
295                   ("p_reflected", self.k_tab[modes] < 0),
296                   ("p_zero_1g", self.k_tab[modes] == 0)):
297     self.observables[name] = (np.abs(ground[:, indices[mask]]) ** 2).sum(axis=1)
298 for n in range(int(self.photon_numbers.max()) + 1):
299     self.observables[f"p_{n}_photon"] = probability[:, self.photon_numbers == n].sum(axis=1)
300 return self.observables
301
302 def observables_dataframe(self, summary=False):
303     """Return the stored population diagnostics as a pandas DataFrame.
304
305     Parameters
306     -----
307     summary : bool, optional
308         False returns the complete history indexed by physical time.
309         True returns observable rows and initial/final columns; final-only
310         storage returns a single final column, never a false initial one.
311
312     Returns
313     -----
314     pandas.DataFrame
315         Raw norm and populations from compute_observables. The summary
316         also contains a time row. No probability is renormalized here.
317
318     Notes
319     -----
320     Requires propagation. Computes observables if absent and leaves the
321     numerical arrays intact; table formatting does not round stored data.
322     """
323     if self.observables is None:
324         self.compute_observables()
325     return observables_table(self.observables, summary, self.store_state)
326
327 def one_photon_wavefunction(self, i, x_tab):
328     """Reconstruct the one-photon, ground-TLS component on a spatial grid.
329
330     Parameters
331     -----
332     i : int
333         Stored output index, not physical time. Standard negative indices are
334         accepted for histories; final-only storage uses its final coefficients.
335     x_tab : array_like of float
336         Spatial coordinates (N_x,) in the periodic box.
337
338     Returns
339     -----
340     numpy.ndarray
341         Complex array (N_x,), phi_1g(x,t)=sum_m C_(e_m,g)(t)*exp(i*k_m*x)/sqrt(L).
342         This sector is not normalized separately; its box-integrated density
343         equals its raw population. With the imposed exp(-i*k*x_tls) annihilation
344         phase, the spatial interaction coordinate is -x_tls.
345     """
346     if self.c_g_array is None:
347         raise RuntimeError("Call propagate_state first")
348     coefficients = self.c_g_array[i] if self.store_state else self.c_g_array
349     indices, modes = self._one_photon_indices()
350     return np.exp(1j * np.outer(x_tab, self.k_tab[modes])) @ coefficients[indices] / \
351         np.sqrt(self.param_atom["L"])
352
353 def _vectors_for(self, t=None):

```

```

354     """Resolve a diagnostic time request to the retained joint ket vectors.
355
356     Parameters
357     -----
358     t : float or None, optional
359         None selects all retained outputs; -1 selects the final state. Other
360         values must lie in [0,T] and select the nearest stored time (no
361         interpolation). Final-only runs accept None, -1 or the final time only.
362
363     Returns
364     -----
365     list[numpy.ndarray]
366         Raw complex vectors (d,), in joint q=2*i+s order. Length N_t for None
367         with a history, otherwise one. Requires propagation; an unavailable
368         earlier state or out-of-range time raises ValueError.
369     """
370     if self.c_g_array is None:
371         raise RuntimeError("Call propagate_state first")
372     if not self.store_state:
373         if t is not None and t != -1 and not np.isclose(t, self.times[-1]):
374             raise ValueError("Only the final state was stored")
375         return [self.result.final_state.full()[:, 0]]
376     if t is None:
377         return [s.full()[:, 0] for s in self.result.states]
378     if t != -1 and not 0 <= t <= self.times[-1]:
379         raise ValueError("Requested time lies outside the stored interval")
380     i = -1 if t == -1 else int(np.argmin(abs(self.times - t)))
381     return [self.result.states[i].full()[:, 0]]
382
383 def compute_atom_density_matrix(self, t=None):
384     """Return the raw field trace rho_TLS(t)=Tr_field |psi(t)><psi(t)|.
385
386     Parameters
387     -----
388     t : float or None, optional
389         None requests all retained samples; -1 requests final; an in-range
390         time uses the nearest retained output. See _vectors_for.
391
392     Returns
393     -----
394     list[numpy.ndarray] or numpy.ndarray
395         None returns K complex matrices (2,2); an explicit time returns one.
396         TLS order is (g,e); rho_(s,s')=sum_i C_(i,s)*C_(i,s').conj().
397         Trace equals the raw squared ket norm. Final-only with None gives a
398         one-element list rather than a reconstructed history.
399     """
400     matrices = [atom_density_matrix(v) for v in self._vectors_for(t)]
401     return matrices if t is None else matrices[0]
402
403 def compute_excited_probability(self, t=None):
404     """Evaluate the raw excited-state population P_e=sum_i |C_(i,e)|**2.
405
406     Parameters
407     -----
408     t : float or None, optional
409         All retained outputs for None, final for -1, or nearest stored output
410         to an in-range physical time. Final-only earlier requests are rejected.
411
412     Returns
413     -----
414     numpy.ndarray or float
415         Real array (K,) for None or scalar for an explicit time. K=N_t with
416         history and 1 otherwise. No ket-norm division is applied.
417     """
418     values = np.array([atom_density_matrix(v)[1, 1].real for v in self._vectors_for(t)])
419     return values if t is None else float(values[0])
420
421 def compute_entropy(self, t=None):
422     """Evaluate the normalized TLS von Neumann entropy in natural logarithms.
423
424     Parameters
425     -----
426     t : float or None, optional
427         Time selector: None for all retained outputs, -1 for final, or nearest
428         stored output to an in-range time; follows _vectors_for.

```

```

429
430 Returns
431 -----
432 numpy.ndarray or float
433     Real array (K,) for None or scalar otherwise. With
434     rho_hat=Tr_field(|psi><psi|)/||psi||**2, S=-Tr(rho_hat*log(rho_hat))
435     lies in [0,log(2)]. For the pure joint state this is field/TLS
436     entanglement entropy. Normalization here removes solver norm drift.
437     """
438     values = np.array([qutip.entropy_vn(qutip.Qobj(atom_density_matrix(v) / np.vdot(v, v).real))
439                        for v in self._vectors_for(t)])
440     return values if t is None else float(values[0])
441
442 def compute_energy(self, t=None):
443     """Evaluate the raw expectation of the same total Hamiltonian used for dynamics.
444
445     Parameters
446     -----
447     t : float or None, optional
448         None for all retained outputs, -1 for final, or nearest retained output
449         to an in-range physical time. No earlier final-only state is available.
450
451     Returns
452     -----
453     numpy.ndarray or float
454         E(t)=Re[psi(t)^dagger*(H0+D*V)*psi(t)], as a real array (K,) for None
455         or scalar otherwise. This retains the raw ket norm and the atomic
456         ground energy convention zero. For static Hermitian H, exact evolution
457         has dE/dt=0; numerical drift is a diagnostic of the solver.
458     """
459     matrix = self.hamiltonian.H0 + self.param_atom["D"] * self.hamiltonian.V
460     values = np.array([np.vdot(v, matrix @ v).real for v in self._vectors_for(t)])
461     return values if t is None else float(values[0])

```

Lines	Mathematical and implementation commentary
1–15	One common engine imports grid, basis, Hamiltonian and diagnostics. <code>momentum_modes</code> remains importable from this module for API convenience.
16–45	Validate finite positive T , dt . Generate nominal output spacing with a roundoff tolerance, always including zero and exact T ; if $\text{dt} > T$, return both endpoints.
46–104	Build configuration from the dictionary API and call the same grid resolver as propagation. Optional selection requires packet information; dimension is evaluated without enumerating a basis.
105–112	Use exact dimensions (13) with the selected count. Reject unknown basis names; no dense allocation is needed to estimate exponential full-space growth.
113–117	Count outputs and estimate $16d$ bytes per ket, retained history depending on storage, two retained vector copies and the sparse-entry bound, (67).
118–129	Return base/selected representations and heuristic feasibility, plus reusable grid/resource DataFrames. Display them through the reporting helper when requested. Excluded memory costs remain in frame attributes; no solver allocation limit is enforced.
130–152	Forward all grid, cap, selection and storage inputs from the dataclass to the dictionary estimator without a separate estimate formula.
153–209	Copy physical dictionaries, resolve base/selected grids, generate output times, build basis and projected Hamiltonian, then prepare the ket. Coefficient arrays are absent until propagation.
210–242	Run <code>sesolve</code> with BDF/default tolerances, raw output normalization and final-state storage. Build a contiguous vector array; alternating ground/excited slices are views, shape (N_t, B) or $(B,)$ for final-only.
243–261	Find tuples with total exactly one, then identify their occupied mode. This works across all three basis orderings and gives an empty list at $N = 0$.

Lines	Mathematical and implementation commentary
262–301	Require propagation, promote final-only arrays internally, and compute raw norm, TLS excitation, signed $1g$ populations and all photon totals. Sum identities are (44).
302–326	Return raw observable histories indexed by time or an endpoint table. Final-only storage yields a final column only. Formatting neither rounds stored data nor renormalizes probabilities.
327–352	The first argument is an output index, not a physical time. Extract only $1g$ amplitudes and multiply by positive Fourier phases and $L^{-1/2}$, (45).
353–382	Resolve diagnostic requests: all retained states for None, final for -1 , or nearest stored output for an in-range time. No interpolation occurs; final-only runs reject earlier requests.
383–402	Reduce each requested raw ket. None returns a list, an explicit time one matrix; final-only with None returns a one-element list.
403–420	Return the real excited diagonal of the raw TLS reduction. None returns an array, an explicit time a scalar; norm drift remains visible.
421–441	Normalize the atomic reduction by the full squared ket norm before natural-log von Neumann entropy. This is (41), bounded by $\log 2$ for normalized valid states.
442–461	Apply the same sparse $H_0 + DV$ and take the real raw expectation. Return a time array or scalar without introducing a second energy-origin convention.

src/reporting.py

```

1  """Tabular presentation of grid, memory and population diagnostics."""
2
3  import pandas as pd
4
5
6  def resource_tables(estimate, config):
7      """Describe the chosen grid and its cost as two reusable DataFrames.
8
9      Parameters
10     -----
11     estimate : dict
12         Numerical resource estimate, including 'grid' with base/selected
13         momentum descriptions. Dimensions count both TLS states.
14     config : ExperimentConfig
15         Inputs used for the estimate; determines which grid control is active.
16
17     Returns
18     -----
19     tables : dict[str, pandas.DataFrame]
20         'grid_table' has one row per base/selected grid, physical bounds,
21         count, zero-mode policy and exact equivalence to the other control.
22         'resource_table' has one row of basis, storage and memory estimates.
23         GiB means bytes / 2**30; one complex128 ket requires 16*d bytes.
24
25     Notes
26     -----
27     Table attributes record requested inputs and excluded memory costs.
28     Effective IR/UV are extrema of retained |k|, not the requested bounds.
29     A missing equivalent M means an explicit symmetric zero-containing grid
30     cannot reproduce that subset. No basis or Hamiltonian is allocated here.
31     """
32     grid = estimate['grid']
33     grid_table = pd.DataFrame.from_dict(
34         {name: {key: value for key, value in grid[name].items()
35                  if key != 'integer_indices'} for name in ('base', 'selected')},
36         orient='index')
37     grid_table.index.name = 'grid'
38     grid_table['equivalent_M'] = grid_table['equivalent_M'].astype('Int64')
39     grid_table.insert(0, 'L', config.param_atom['L'])
40     grid_table.insert(1, 'delta_k', grid['delta_k'])
41     grid_table.insert(2, 'control', 'explicit M' if config.CTRL_M_EXPLICIT else 'IR/UV')
42     grid_table.insert(3, 'requested_M', config.M if config.CTRL_M_EXPLICIT else pd.NA)

```

```

43     cutoffs = config.cutoffs if not config.CTRL_M_EXPLICIT else None
44     grid_table.insert(4, 'requested_IR', cutoffs['ir_cutoff'] if cutoffs else pd.NA)
45     grid_table.insert(5, 'requested_UV', cutoffs['uv_cutoff'] if cutoffs else pd.NA)
46     grid_table.attrs['integer_indices'] = {
47         name: grid[name]['integer_indices'] for name in ('base', 'selected')}
48     resource_table = pd.DataFrame([
49         'basis': config.truncation, 'n_max': config.n_max,
50         'store_state': config.store_state,
51         **{key: estimate[key] for key in (
52             'n_modes', 'dimension', 'n_times', 'ket_gib', 'history_gib',
53             'retained_vectors_gib', 'hamiltonian_nnz_bound', 'feasible')}}])
54     resource_table.attrs['memory_excludes'] = (
55         'Sparse matrices, basis/Python objects, solver workspace and plots.')
56     resource_table.attrs['feasibility_rule'] = (
57         'dimension <= 100000 and retained_vectors_gib <= 1; heuristic only.')
58     return {'grid_table': grid_table, 'resource_table': resource_table}
59
60
61 def display_resource_tables(tables):
62     """Display resource DataFrames in a notebook or as terminal tables.
63
64     Parameters
65     -----
66     tables : dict[str, pandas.DataFrame]
67         Mapping containing 'grid_table' and 'resource_table'; an estimate
68         returned by resource_estimation also satisfies this contract.
69
70     Returns
71     -----
72     None
73         Displays two HTML-capable tables in an active IPython shell, or prints
74         their tabular text in a terminal. The input frames are not modified.
75     """
76     try:
77         from IPython import get_ipython
78         from IPython.display import display
79         interactive = get_ipython() is not None
80     except ImportError:
81         interactive = False
82     for name in ('grid_table', 'resource_table'):
83         if interactive:
84             display(tables[name])
85         else:
86             print(tables[name].to_string())
87
88
89 def observables_table(observables, summary=False, initial_available=True):
90     """Convert raw time-series diagnostics to a history or endpoint table.
91
92     Parameters
93     -----
94     observables : dict[str, numpy.ndarray]
95         Equal-length real arrays from Experiment.compute_observables, with
96         'time', squared ket norm and raw sector populations.
97     summary : bool, optional
98         False returns all samples; True returns one row per observable and
99         columns 'initial'/'final' when the initial state was stored.
100     initial_available : bool, optional
101         False identifies final-only storage, so a summary has only 'final'.
102
103     Returns
104     -----
105     frame : pandas.DataFrame
106         History: shape (K, J), indexed by time, with J observable columns.
107         Summary: shape (J+1, 2), or (J+1, 1) for final-only runs; includes time
108         as an observable row. Values retain solver norm drift and precision.
109     """
110     frame = pd.DataFrame(observables)
111     if not summary:
112         return frame.set_index('time')
113     if initial_available:
114         result = frame.iloc[[0, -1]].T
115         result.columns = ['initial', 'final']
116     else:
117         result = frame.iloc[[-1]].T

```

```

118     result.columns = ['final']
119     result.index.name = 'observable'
120     return result

```

Lines	Mathematical and implementation commentary
6–58	resource_tables: Create grid and resource DataFrames from the numerical estimate. Preserve requested controls, effective cutoffs and exact equivalent counts; nullable integer counts distinguish unavailable equivalence. Frame attributes retain integer indices and memory exclusions.
61–86	display_resource_tables: Display HTML-capable DataFrames in an active IPython shell, otherwise terminal tables. The input frames remain reusable and unmodified.
89–120	observables_table: Convert raw diagnostics to a time-indexed history or observable-by-endpoint frame. Initial/final columns require initial availability; final-only storage produces one final column. Numerical precision and norm drift are preserved.

```

experiment/_common.py
1  """Shared experiment execution, CLI inputs and exports."""
2
3  import argparse
4  from dataclasses import asdict
5  import json
6  from pathlib import Path
7
8  import numpy as np
9
10 from src.experiment import Experiment, estimate_config
11 from src.xp_config import ExperimentConfig
12
13
14 def run(config, progress=False):
15     """Build, propagate and diagnose one configuration using the shared engine.
16
17     Parameters
18     -----
19     config : ExperimentConfig
20         Complete physical, basis, grid and solver inputs; not modified here.
21     progress : bool, optional
22         Show the solver's progress bar when True.
23
24     Returns
25     -----
26     Experiment
27         Propagated object with result, coefficient arrays and raw observables.
28         Storage follows config.store_state; no plotting or export occurs.
29     """
30     experiment = Experiment(config)
31     experiment.propagate_state(progress=progress)
32     experiment.compute_observables()
33     return experiment
34
35
36 def save_arrays(path, config, **arrays):
37     """Save numerical experiment arrays with their input configuration in NPZ.
38
39     Parameters
40     -----
41     path : str or pathlib.Path
42         Destination archive; parent directories are created if needed.
43     config : ExperimentConfig
44         Base configuration serialized as JSON under 'configuration_json'.
45     **arrays : array_like
46         Named numerical arrays or NumPy-compatible metadata to store.
47
48     Returns
49     -----

```

```

50     None
51     Writes/overwrites the archive. Non-JSON configuration values (such as
52     complex alpha) stringify; sweep-specific inputs belong in the arrays.
53     """
54     path = Path(path)
55     path.parent.mkdir(parents=True, exist_ok=True)
56     metadata = json.dumps(asdict(config), default=lambda value: str(value))
57     np.savez(path, configuration_json=metadata, **arrays)
58
59
60 def parser(description):
61     """Create the shared command-line parser in the current physical conventions.
62
63     Parameters
64     -----
65     description : str
66         Module description displayed by --help.
67
68     Returns
69     -----
70     argparse.ArgumentParser
71         Parser for D, L, omega_0, x_tls, sqrt/flat profile, grid control, packet
72         preparation, TLS state, photon cap, T/dt, RWA, output path and progress.
73         It does not parse arguments yet. Coupling defaults to sqrt; cutoff
74         control is the default, and --ctrl-m-explicit activates the M input.
75     """
76     p = argparse.ArgumentParser(description=description)
77     p.add_argument('--D', type=float, default=0.2)
78     p.add_argument('--L', type=float, default=2 * np.pi)
79     p.add_argument('--omega-0', type=float, default=1.0)
80     p.add_argument('--x-tls', type=float, default=0.0)
81     p.add_argument('--coupling', choices=('sqrt', 'flat'), default='sqrt')
82     p.add_argument('--ctrl-m-explicit', action=argparse.BooleanOptionalAction, default=False)
83     p.add_argument('--M', type=int, default=5)
84     p.add_argument('--ir-cutoff', type=float, default=0.0)
85     p.add_argument('--uv-cutoff', type=float, default=2.0)
86     p.add_argument('--k-0', type=float, default=1.0)
87     p.add_argument('--sigma-k', type=float, default=0.4)
88     p.add_argument('--x-0', type=float, default=-2.0)
89     p.add_argument('--photon-state', choices=('number', 'coherent'), default='number')
90     p.add_argument('--n', type=int, default=1)
91     p.add_argument('--alpha', type=complex, default=1.0)
92     p.add_argument('--atom-state', choices=('g', 'e', '+', '-'), default='g')
93     p.add_argument('--n-max', type=int, default=2)
94     p.add_argument('--truncation', choices=('full', 'full+totalcap', 'truncated'), default='full+totalcap')
95     p.add_argument('--T', type=float, default=4.0)
96     p.add_argument('--dt', type=float, default=0.05)
97     p.add_argument('--RWA', action='store_true')
98     p.add_argument('--out', type=Path)
99     p.add_argument('--progress', action='store_true')
100    return p
101
102
103 def config_from_args(args):
104     """Convert parsed shared CLI inputs into the dictionary-based configuration.
105
106     Parameters
107     -----
108     args : argparse.Namespace
109         Values obtained from parser, possibly extended by an experiment module.
110
111     Returns
112     -----
113     ExperimentConfig
114         Three physical/solver dictionaries and basis/grid settings, with
115         store_state=True by default. Both M and cutoffs are retained, but only
116         the active control defines the grid. No historical g-to-D conversion
117         is inferred from numerical values supplied by the user.
118     """
119    return ExperimentConfig(
120        param_photon={'k_0': args.k_0, 'sigma_k': args.sigma_k, 'x_0': args.x_0,
121                     'state': args.photon_state, 'n': args.n, 'alpha': args.alpha},
122        param_atom={'omega_0': args.omega_0, 'D': args.D, 'L': args.L,
123                   'x_tls': args.x_tls, 'coupling': args.coupling, 'initial_state': args.atom_state},
124        param_timeevol={'T': args.T, 'dt': args.dt},

```

```

125         cutoffs={'ir_cutoff': args.ir_cutoff, 'uv_cutoff': args.uv_cutoff},
126         n_max=args.n_max, truncation=args.truncation, RWA=args.RWA,
127         CTRL_M_EXPLICIT=args.ctrl_m_explicit, M=args.M)
128
129
130 def finish(fig, args, config, **arrays):
131     """Display a completed figure or export it alongside numerical arrays.
132
133     Parameters
134     -----
135     fig : matplotlib.figure.Figure
136         Figure returned by a plot function.
137     args : argparse.Namespace
138         Contains 'out' (pathlib.Path or None) from the shared parser.
139     config : ExperimentConfig
140         Configuration included in the corresponding NPZ archive.
141     **arrays : array_like
142         Named arrays/metadata to export when out is supplied.
143
144     Returns
145     -----
146     None
147         With out, creates parents, saves the figure at 150 dpi and a same-stem
148         .npz archive, then closes the figure. Otherwise calls pyplot.show().
149     """
150     import matplotlib.pyplot as plt
151     if args.out:
152         args.out.parent.mkdir(parents=True, exist_ok=True)
153         fig.savefig(args.out, dpi=150)
154         save_arrays(args.out.with_suffix('.npz'), config, **arrays)
155         plt.close(fig)
156         print(f'Saved {args.out}')
157     else:
158         plt.show()

```

Lines	Mathematical and implementation commentary
14–33	run : Construct one Experiment , propagate, then compute observables in that order. Return the numerical object so notebook analyses can reuse its kets and Hamiltonian.
36–57	save_arrays : Create the output directory and write NPZ arrays plus configuration_json from the base dataclass. Non-JSON values stringify; ordinary numerical arrays can be loaded without pickle.
60–100	parser : Shared CLI names use $D, L, M, k_0, \sigma_k, x_0, N, T$. Default profile is sqrt . Explicit M requires its boolean flag; default CLI grid control uses cutoffs.
103–127	config_from_args : Construct the three dictionaries and dataclass with the current conventions. Both count and cutoff inputs may be stored, but only the selected control is active; historical values are not automatically converted.
130–158	finish : With an output path, save the figure and same-stem NPZ, then close it. Otherwise display the figure. Backend selection remains the environment or notebook responsibility.

```

experiment/atom_evolution.py
1  """TLS population and entropy for several photon bases."""
2
3  from dataclasses import replace
4
5  from ._common import config_from_args, finish, parser, run
6
7
8  def run_atom_evolution(config, schemes=('full+totalcap', 'truncated'), progress=False):
9      """Propagate one TLS preparation under several photon-space constraints.
10
11      Parameters
12      -----
13      config : ExperimentConfig
14          Shared physical inputs and photon cap; input is not modified.

```

```

15     schemes : iterable[str], optional
16         Basis names; default ('full+totalcap', 'truncated').
17     progress : bool, optional
18         Show solver progress for each trajectory.
19
20     Returns
21     -----
22     dict[str, Experiment]
23         Scheme -> propagated object, forcing store_state=True in each copy.
24         The retained histories support raw  $P_e(t) = \sum_i |C_{i,e}|^2$  and
25          $S_{\text{TLS}}(t) = -\text{Tr}(\rho_{\text{hat}} \log(\rho_{\text{hat}}))$ , with  $\rho_{\text{hat}}$  normalized to trace one.
26     """
27     return {scheme: run(replace(config, truncation=scheme, store_state=True), progress)
28             for scheme in schemes}
29
30
31 def plot_atom_evolution(runs):
32     """Plot raw TLS excitation and normalized entanglement entropy for each basis.
33
34     Parameters
35     -----
36     runs : dict[str, Experiment]
37         Propagated full-history objects returned by run_atom_evolution.
38
39     Returns
40     -----
41     matplotlib.figure.Figure
42         Two panels of  $P_e(t)$  and  $S_{\text{TLS}}(t)$ , using natural-log entropy. Builds
43         the figure without showing, exporting or rerunning the trajectories.
44     """
45     import matplotlib.pyplot as plt
46     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
47     for scheme, experiment in runs.items():
48         axes[0].plot(experiment.times, experiment.compute_excited_probability(), label=scheme)
49         axes[1].plot(experiment.times, experiment.compute_entropy(), label=scheme)
50     axes[0].set(xlabel='t', ylabel=r'$P_e$')
51     axes[1].set(xlabel='t', ylabel=r'$S_{\mathrm{TLS}}$')
52     for ax in axes:
53         ax.legend()
54     fig.tight_layout()
55     return fig
56
57
58 def main(argv=None):
59     """Run per-basis TLS evolution and export excitation curves.
60
61     Parameters
62     -----
63     argv : list[str] or None, optional
64         CLI arguments without the program name. None reads sys.argv[1:].
65
66     Returns
67     -----
68     None
69         Parses parameters, displays resource tables, simulates and plots.
70         --out saves the figure plus a same-stem NPZ; otherwise displays it.
71         Argument parsing may raise SystemExit for --help or invalid input.
72     """
73     p = parser(__doc__)
74     p.add_argument('--schemes', default='full+totalcap,truncated')
75     args = p.parse_args(argv)
76     config = config_from_args(args)
77     from src.experiment import estimate_config
78     for scheme in args.schemes.split(','):
79         estimate_config(replace(config, truncation=scheme))
80     runs = run_atom_evolution(config, args.schemes.split(','), args.progress)
81     arrays = {f'{scheme}_P_e': exp.compute_excited_probability() for scheme, exp in runs.items()}
82     finish(plot_atom_evolution(runs), args, config, time=next(iter(runs.values())).times, **arrays)
83
84
85 if __name__ == '__main__':
86     main()

```

Lines	Mathematical and implementation commentary
8–28	<code>run_atom_evolution</code> : Replace the configuration by each scheme and force history storage. Return a dictionary of experiments, so excitation, entropy and notebook fidelity derive from the same stored trajectories.
31–55	<code>plot_atom_evolution</code> : Plot raw P_e and normalized TLS entropy against output times for every returned scheme. This plot does not itself propagate or compute cross-state fidelity.
58–82	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

experiment/compare_mode_selection.py

```

1 """Truncation and mode-selection errors relative to one finite trajectory."""
2
3 from dataclasses import replace
4
5 import numpy as np
6
7 from src.fidelities import fidelities_over_time
8 from ._common import config_from_args, finish, parser, run
9
10
11 def run_mode_selection(config, D_values=(0.02, 0.1, 0.2),
12                        schemes=('truncated', 'full+totalcap'),
13                        photon_windows=(0.5, 1.5, 3.0), atom_windows=(0.0, 0.5, 1.5),
14                        D_heatmap=None, progress=False):
15     """Measure truncation and selected-grid differences from an unselected run.
16
17     Parameters
18     -----
19     config : ExperimentConfig
20         Shared inputs, requiring store_state=True. The simulated comparator
21         always uses the unselected 'full+totalcap' basis at the same D.
22     D_values : array_like of float, optional
23         Couplings (Q,) for the selection-on/off sweep.
24     schemes : iterable[str], optional
25         S candidate photon bases, default ('truncated', 'full+totalcap').
26     photon_windows : array_like of float, optional
27         P nonnegative packet-window radii w_p in units of sigma_k.
28     atom_windows : array_like of float, optional
29         A nonnegative resonance-window radii w_a in units of sigma_k.
30     D_heatmap : float or None, optional
31         Fixed coupling for the window scan; None uses config.param_atom['D'].
32     progress : bool, optional
33         Show solver progress for each trajectory.
34
35     Returns
36     -----
37     dict[str, object]
38         D (Q,), schemes (S,), selection flags [False,True], window arrays
39         (P,)/(A,), and D_heatmap. F_state/F_atom and their _initial arrays
40         have shape (S,2,Q), ordered by scheme, selection flag, then coupling.
41         Mean-only _heatmap arrays have shape (S,P,A). Means are arithmetic
42         averages over stored output times. Missing selected modes are vacuum
43         in the common full embedding, and selected preparations are normalized
44         on their own grids, so initial fidelities can already be below one.
45     """
46     if not config.store_state:
47         raise ValueError('Time-mean comparisons require store_state=True')
48     D_values = np.asarray(D_values, dtype=float)
49     schemes = tuple(schemes)
50     pw, aw = np.asarray(photon_windows), np.asarray(atom_windows)
51     shape = (len(schemes), 2, len(D_values))
52     metrics = {name: np.empty(shape) for name in ('F_state', 'F_atom', 'F_state_initial', 'F_atom_initial')}
53     for i, D in enumerate(D_values):
54         current = config.with_D(float(D))
55         reference = run(replace(current, truncation='full+totalcap', mode_selection=False), progress)
56         for j, scheme in enumerate(schemes):
57             for selected in (False, True):

```

```

58         candidate = run(replace(current, truncation=scheme, mode_selection=selected), progress)
59         values = fidelities_over_time(candidate, reference)
60         for name, series in values.items():
61             metrics[name][j, int(selected), i] = np.mean(series)
62             metrics[f'{name}_initial'][j, int(selected), i] = series[0]
63     current = config.with_D(config.param_atom['D'] if D_heatmap is None else D_heatmap)
64     reference = run(replace(current, truncation='full+totalcap', mode_selection=False), progress)
65     heatmaps = {name: np.empty((len(schemes), len(pw), len(aw))) for name in ('F_state', 'F_atom')}
66     for j, scheme in enumerate(schemes):
67         for x, photon_window in enumerate(pw):
68             for y, atom_window in enumerate(aw):
69                 candidate = run(replace(current, truncation=scheme, mode_selection=True,
70                                     photon_window=float(photon_window), atom_window=float(atom_window)),
71                               progress)
72                 for name, series in fidelities_over_time(candidate, reference).items():
73                     heatmaps[name][j, x, y] = np.mean(series)
74     return {'D': D_values, 'schemes': schemes, 'selection': np.array([False, True]),
75           'photon_windows': pw, 'atom_windows': aw, 'D_heatmap': current.param_atom['D'],
76           **metrics, **{f'{name}_heatmap': value for name, value in heatmaps.items()}}
77
78 def plot_mode_selection(results):
79     """Plot selection-on/off sweeps and fixed-D window scans for both fidelities.
80
81     Parameters
82     -----
83     results : dict[str, object]
84         run_mode_selection output with sweep arrays (S,2,Q) and maps (S,P,A).
85
86     Returns
87     -----
88     matplotlib.figure.Figure
89         (S+1)-by-2 panels: top row shows sample-mean fidelity against D;
90         remaining rows show one basis per window map. Map slices are transposed
91         for horizontal photon window and vertical atom window coordinates.
92     """
93     import matplotlib.pyplot as plt
94     schemes = results['schemes']
95     fig, axes = plt.subplots(len(schemes) + 1, 2, figsize=(11, 3 * (len(schemes) + 1)), squeeze=False)
96     for column, name in enumerate(('F_state', 'F_atom')):
97         ax = axes[0, column]
98         for j, scheme in enumerate(schemes):
99             for selected in (False, True):
100                 ax.plot(results['D'], results[name][j, int(selected)],
101                        'o-' if selected else 'o--', label=f'{scheme}, selection={selected}')
102         ax.set(xlabel='D', ylabel=f'time mean {name}')
103         ax.legend(fontsize=8)
104         for j, scheme in enumerate(schemes):
105             ax = axes[j + 1, column]
106             image = ax.pcolormesh(results['photon_windows'], results['atom_windows'],
107                                results[f'{name}_heatmap'][j].T, shading='nearest', vmin=0, vmax=1)
108             fig.colorbar(image, ax=ax)
109             ax.set(xlabel='packet window / sigma_k', ylabel='resonance window / sigma_k',
110                  title=f'{scheme}: {name}')
111     fig.tight_layout()
112     return fig
113
114
115 def main(argv=None):
116     """Run selection and window sweeps and export fidelity arrays.
117
118     Parameters
119     -----
120     argv : list[str] or None, optional
121         CLI arguments without the program name. None reads sys.argv[1:].
122
123     Returns
124     -----
125     None
126         Parses parameters, displays resource tables, simulates and plots.
127         --out saves the figure plus a same-stem NPZ; otherwise displays it.
128         Argument parsing may raise SystemExit for --help or invalid input.
129     """
130     p = parser(__doc__)
131     p.add_argument('--D-values', default='0.02,0.1,0.2')

```

```

132 p.add_argument('--photon-windows', default='0.5,1.5,3.0')
133 p.add_argument('--atom-windows', default='0.0,0.5,1.5')
134 args = p.parse_args(argv)
135 config = config_from_args(args)
136 from src.experiment import estimate_config
137 estimate_config(config)
138 estimate_config(replace(config, mode_selection=True))
139 results = run_mode_selection(config, [float(v) for v in args.D_values.split(',')],
140                                photon_windows=[float(v) for v in args.photon_windows.split(',')],
141                                atom_windows=[float(v) for v in args.atom_windows.split(',')], progress=args.
142                                progress)
143 finish(plot_mode_selection(results), args, config, **results)
144
145 if __name__ == '__main__':
146     main()

```

Lines	Mathematical and implementation commentary
11–75	<code>run_mode_selection</code> : Use an unselected total-cap comparator at each coupling. Sweep schemes and both selection flags; return $(S, 2, Q)$ means/initial metrics. At one fixed D , scan windows to return (S, P, A) mean-only heatmaps.
78–112	<code>plot_mode_selection</code> : Plot selection on/off sweeps and window maps for both metrics. Transpose the stored (P, A) slice for plotting so the horizontal axis remains photon window and vertical axis atom window.
115–142	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

```

experiment/full_cumulative_fidelity.py
1 """Final-state convergence between neighbouring total photon caps."""
2
3 from dataclasses import replace
4
5 import numpy as np
6
7 from src.fidelities import compare_states
8 from ._common import config_from_args, finish, parser, run
9
10
11 def run_cap_convergence(config, caps=(1, 2, 3, 4), progress=False):
12     """Compare neighboring total-photon caps in the implicit ambient full basis.
13
14     Parameters
15     -----
16     config : ExperimentConfig
17         Shared preparation and dynamics; basis/storage are replaced in copies.
18     caps : iterable[int], optional
19         At least two strictly increasing nonnegative caps  $N_j$ ; defaults 1..4.
20         Number preparation must fit every cap; coherent input is projected
21         and normalized independently at each cap.
22     progress : bool, optional
23         Show solver progress.
24
25     Returns
26     -----
27     dict[str, object]
28         'caps' integer array (C,), 'runs' list of C propagated Experiments;
29         F_state/F_atom and their _initial counterparts are float arrays (C-1,).
30         Element j compares  $N_j$  with  $N_{(j+1)}$ , at T or zero respectively. All
31         trajectories use 'full+totalcap' and store only the final evolved ket.
32         Squared fidelities include initial projection differences; no common
33         intersection normalization or ambient full-space allocation is used.
34     """
35     caps = list(caps)
36     if len(caps) < 2 or any(b <= a for a, b in zip(caps, caps[1:])):
37         raise ValueError('At least two strictly increasing caps are required')
38     runs = [run(replace(config, n_max=cap, truncation='full+totalcap', store_state=False), progress)
39             for cap in caps]

```

```

40 final, initial = [], []
41 for a, b in zip(runs, runs[1:]):
42     final.append(compare_states(a.result.final_state, a.basis, a.k_tab,
43                               b.result.final_state, b.basis, b.k_tab))
44     initial.append(compare_states(a.state0, a.basis, a.k_tab, b.state0, b.basis, b.k_tab))
45 return {'caps': np.array(caps), 'runs': runs,
46        **{name: np.array([row[name] for row in final]) for name in ('F_state', 'F_atom')},
47        **{f'{name}_initial': np.array([row[name] for row in initial])
48          for name in ('F_state', 'F_atom')}}
49
50
51 def plot_cap_convergence(results):
52     """Plot initial and final squared fidelities between adjacent requested caps.
53
54     Parameters
55     -----
56     results : dict[str, object]
57         run_cap_convergence output; caps (C,), metric arrays (C-1,).
58
59     Returns
60     -----
61     matplotlib.figure.Figure
62         F_state and F_atom panels against the lower cap N_j of each compared
63         pair. Solid curves are final metrics; dashed curves are initial ones.
64         Consecutive requested caps need not differ by exactly one.
65
66     """
67     import matplotlib.pyplot as plt
68     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
69     for ax, name in zip(axes, ('F_state', 'F_atom')):
70         ax.plot(results['caps'][:-1], results[name], 'o-', label='final')
71         ax.plot(results['caps'][:-1], results[f'{name}_initial'], 'o--', label='initial')
72         ax.set(xlabel='lower cap of compared pair', ylabel=name, ylim=(0, 1.05))
73         ax.legend()
74     fig.tight_layout()
75     return fig
76
77 def main(argv=None):
78     """Run total-cap convergence and export initial/final fidelities.
79
80     Parameters
81     -----
82     argv : list[str] or None, optional
83         CLI arguments without the program name. None reads sys.argv[1:].
84
85     Returns
86     -----
87     None
88
89     Parses parameters, displays resource tables, simulates and plots.
90     --out saves the figure plus a same-stem NPZ; otherwise displays it.
91     Argument parsing may raise SystemExit for --help or invalid input.
92
93     """
94     p = parser(__doc__)
95     p.set_defaults(photon_state='coherent')
96     p.add_argument('--caps', default='1,2,3,4')
97     args = p.parse_args(argv)
98     config = config_from_args(args)
99     caps = [int(value) for value in args.caps.split(',') ]
100     from src.experiment import estimate_config
101     estimate_config(replace(config, n_max=max(caps), truncation='full+totalcap', store_state=False))
102     results = run_cap_convergence(config, caps, args.progress)
103     finish(plot_cap_convergence(results), args, config,
104            **{key: value for key, value in results.items() if key != 'runs'})
105
106 if __name__ == '__main__':
107     main()

```

Lines	Mathematical and implementation commentary
11–48	run_cap_convergence: Require at least two strictly increasing caps; run total-cap final-only trajectories at each. Compare adjacent list entries initially and finally using physical embedding; retain all run objects.
51–74	plot_cap_convergence: Two metric panels distinguish initial and final fidelities. The horizontal value is the lower cap in each adjacent pair, not necessarily a comparison of N with $N + 1$.
77–102	main: Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

experiment/scattering.py

```

1  """Gaussian photon scattering, with the user's original dictionary API."""
2
3  import csv
4  from pathlib import Path
5
6  import numpy as np
7
8  from src.xp_config import ExperimentConfig
9  from .common import config_from_args, finish, parser, run
10
11
12 def run_scattering(param_photon, param_atom, param_time_evol, cutoffs=None,
13                   n_max=3, truncation='full+totalcap', RWA=False, store_state=True,
14                   store_results=False, progress=False, CTRL_M_EXPLICIT=False, M=None):
15     """Propagate Gaussian-photon scattering through the original dictionary API.
16
17     Parameters
18     -----
19     param_photon : dict[str, object]
20         k_0, sigma_k, x_0 and optional state/n/alpha preparation inputs.
21     param_atom : dict[str, object]
22         omega_0, D, L, x_tls, coupling ('sqrt'/'flat') and initial_state.
23     param_time_evol : dict[str, object]
24         T/dt and optional method/rtol/atol solver settings.
25     cutoffs : dict[str, float] or None, optional
26         ir_cutoff and uv_cutoff when explicit-M control is inactive.
27     n_max : int, optional
28         Nonnegative photon cap, default 3.
29     truncation : str, optional
30         Retained photon basis, default 'full+totalcap'.
31     RWA : bool, optional
32         Keep only excitation-conserving terms when True.
33     store_state : bool, optional
34         Retain output history (True) or only the final ket (False).
35     store_results : bool, optional
36         Write/overwrite repository results/scattering.csv with raw observables.
37     progress : bool, optional
38         Show solver progress.
39     CTRL_M_EXPLICIT : bool, optional
40         Choose exact odd M when True; choose IR/UV bounds otherwise.
41     M : int or None, optional
42         Explicit base-grid count, ignored under cutoff control.
43
44     Returns
45     -----
46     Experiment
47         Propagated object with raw observables, kets and diagnostics. Positive,
48         negative and zero-momentum packet amplitudes are all retained.
49         observables_dataframe(summary=True) exposes the endpoint table.
50     """
51     config = ExperimentConfig(param_photon, param_atom, param_time_evol, cutoffs,
52                             n_max, truncation, RWA, store_state,
53                             CTRL_M_EXPLICIT=CTRL_M_EXPLICIT, M=M)
54     experiment = run(config, progress)
55     if store_results:
56         path = Path(__file__).resolve().parents[1] / 'results' / 'scattering.csv'
57         path.parent.mkdir(parents=True, exist_ok=True)

```

```

58         with path.open('w', newline='') as handle:
59             writer = csv.writer(handle)
60             writer.writerow(experiment.observables)
61             writer.writerows(zip(*experiment.observables.values()))
62     return experiment
63
64
65 def plot_scattering(experiment):
66     """Plot the one-photon spatial density and raw scattering-sector populations.
67
68     Parameters
69     -----
70     experiment : Experiment
71         Propagated object with compute_observables already called; history
72         gives initial/middle/final snapshots, final-only gives one snapshot.
73
74     Returns
75     -----
76     matplotlib.figure.Figure
77         Two panels:  $|\phi_{1g}(x,t)|^2$  over one periodic box and populations
78         versus time. Fourier reconstruction uses  $\exp(+ikx)/\sqrt{L}$ , so the
79          $\exp(-ikx_{tls})$  Hamiltonian convention places the marker at  $-x_{tls}$ .
80         Directional populations describe the one-photon, ground-TLS sector;
81         they are not divided by incident flux or total ket norm.
82     """
83     import matplotlib.pyplot as plt
84     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
85     times = experiment.observables['time']
86     x = np.linspace(-experiment.param_atom['L'] / 2, experiment.param_atom['L'] / 2, 500, endpoint=False)
87     for i in sorted({0, len(times) // 2, len(times) - 1}):
88         phi = experiment.one_photon_wavefunction(i, x)
89         axes[0].plot(x, abs(phi) ** 2, label=f't={times[i]:.2f}')
90         # With  $u \sim \exp(-ikx_{tls})$  and  $\phi \sim \exp(+ikx)$ , the spatial coupling is at  $-x_{tls}$ .
91         axes[0].axvline(-experiment.param_atom['x_tls'], color='black', linestyle=':',
92                        label='interaction coordinate ( $-x_{tls}$ )')
93         axes[0].set(xlabel='x', ylabel=r'$|\phi_{1g}(x,t)|^2$')
94         for name in ('p_transmitted', 'p_reflected', 'p_zero_1g', 'p_excited', 'norm'):
95             axes[1].plot(times, experiment.observables[name], label=name)
96         axes[1].set(xlabel='t', ylabel='population')
97     for ax in axes:
98         ax.legend()
99     fig.tight_layout()
100    return fig
101
102
103 def main(argv=None):
104     """Run dictionary-API scattering and export raw population curves.
105
106     Parameters
107     -----
108     argv : list[str] or None, optional
109         CLI arguments without the program name. None reads sys.argv[1:].
110
111     Returns
112     -----
113     None
114         Parses parameters, displays resource tables, simulates and plots.
115         --out saves the figure plus a same-stem NPZ; otherwise displays it.
116         Argument parsing may raise SystemExit for --help or invalid input.
117     """
118     p = parser(__doc__)
119     args = p.parse_args(argv)
120     config = config_from_args(args)
121     from src.experiment import estimate_config
122     estimate_config(config)
123     experiment = run_scattering(config.param_photon, config.param_atom, config.param_time_evol,
124                                config.cutoffs, config.n_max, config.truncation, config.RWA,
125                                progress=args.progress, CTRL_M_EXPLICIT=config.CTRL_M_EXPLICIT, M=config.M)
126     finish(plot_scattering(experiment), args, config, **experiment.observables)
127
128
129 if __name__ == '__main__':
130     main()

```

Lines	Mathematical and implementation commentary
12–62	<code>run_scattering</code> : Preserve the user dictionary API and append grid-control inputs. Run shared dynamics; optional CSV contains observable columns only and overwrites its fixed repository results path.
65–100	<code>plot_scattering</code> : Plot initial/middle/final $1g$ Fourier densities and directional/zero/TLS/norm populations. The interaction marker is $-x_{\text{tls}}$, derived in (47).
103–126	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

experiment/sweep_D_fidelity.py

```

1 """Time-mean state/TLS fidelities versus the physical coupling D."""
2
3 from dataclasses import replace
4
5 import numpy as np
6
7 from src.fidelities import fidelities_over_time
8 from ._common import config_from_args, finish, parser, run
9
10
11 def run_coupling_sweep(config, D_values, schemes=('truncated',), reference='full+totalcap', progress=False):
12     """Sweep physical D and compare each candidate with a simulated trajectory.
13
14     Parameters
15     -----
16     config : ExperimentConfig
17         Shared inputs; D and basis are changed in copies, with histories forced.
18     D_values : array_like of float
19         Nonempty finite vector (Q,) of physical couplings in  $H=H_0+D\cdot V$ .
20     schemes : iterable[str], optional
21         S candidate basis names, default ('truncated',).
22     reference : str, optional
23         Simulated comparator basis, default 'full+totalcap'. This choice is
24         separate from the implicit full basis used to calculate overlaps.
25     progress : bool, optional
26         Show solver progress for each trajectory.
27
28     Returns
29     -----
30     dict[str, object]
31         D array (Q,), schemes tuple (S,), reference string, and four float
32         arrays (S,Q): F_state/F_atom are arithmetic sample means
33         sum_j F(t_j)/N_t; _initial arrays contain F(0). These are neither final
34         fidelities nor quadrature approximations to an integral time average.
35         Identical candidate/comparator schemes reuse the comparator run.
36     """
37     D_values = np.asarray(D_values, dtype=float)
38     if D_values.ndim != 1 or len(D_values) == 0 or not np.isfinite(D_values).all():
39         raise ValueError('D_values must be a nonempty finite one-dimensional array')
40     schemes = tuple(schemes)
41     shape = (len(schemes), len(D_values))
42     out = {'D': D_values, 'schemes': schemes, 'reference': reference,
43           **{name: np.empty(shape) for name in
44              ('F_state', 'F_atom', 'F_state_initial', 'F_atom_initial')}}
45     for i, D in enumerate(D_values):
46         at_D = replace(config.with_D(float(D)), store_state=True)
47         ref = run(replace(at_D, truncation=reference), progress)
48         for j, scheme in enumerate(schemes):
49             candidate = ref if scheme == reference else run(replace(at_D, truncation=scheme), progress)
50             fidelities = fidelities_over_time(candidate, ref)
51             for name, values in fidelities.items():
52                 out[name][j, i] = np.mean(values)
53                 out[f'{name}_initial'][j, i] = values[0]
54     return out
55
56
57 def plot_coupling_sweep(results):

```

```

58 """Plot sample-mean and initial squared fidelities versus physical D.
59
60 Parameters
61 -----
62 results : dict[str, object]
63     run_coupling_sweep output with D (Q,) and metrics (S,Q).
64
65 Returns
66 -----
67 matplotlib.figure.Figure
68     Two metric panels; one solid mean curve per scheme and dashed initial
69     curves. The figure is returned without display or export.
70 """
71 import matplotlib.pyplot as plt
72 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
73 for ax, name in zip(axes, ('F_state', 'F_atom')):
74     for j, scheme in enumerate(results['schemes']):
75         ax.plot(results['D'], results[name][j], 'o-', label=scheme)
76         ax.plot(results['D'], results[f'{name}_initial'][j], '--', alpha=0.5)
77     ax.set(xlabel='D', ylabel=f'time mean {name}', ylim=(0, 1.05))
78     ax.legend()
79 fig.tight_layout()
80 return fig
81
82
83 def main(argv=None):
84     """Run the physical-D sweep and export sample-mean/initial metrics.
85
86     Parameters
87     -----
88     argv : list[str] or None, optional
89         CLI arguments without the program name. None reads sys.argv[1:].
90
91     Returns
92     -----
93     None
94         Parses parameters, displays resource tables, simulates and plots.
95         --out saves the figure plus a same-stem NPZ; otherwise displays it.
96         Argument parsing may raise SystemExit for --help or invalid input.
97     """
98     p = parser(__doc__)
99     p.add_argument('--D-values', default='0.02,0.1,0.2')
100    p.add_argument('--schemes', default='truncated')
101    p.add_argument('--reference', choices=('full', 'full+totalcap', 'truncated'), default='full+totalcap')
102    args = p.parse_args(argv)
103    config = config_from_args(args)
104    from src.experiment import estimate_config
105    for scheme in set(args.schemes.split(',') | {args.reference}):
106        estimate_config(replace(config, truncation=scheme))
107    results = run_coupling_sweep(config, [float(v) for v in args.D_values.split(',')],
108                                   args.schemes.split(','), args.reference, args.progress)
109    finish(plot_coupling_sweep(results), args, config, **results)
110
111
112 if __name__ == '__main__':
113     main()

```

Lines	Mathematical and implementation commentary
11–54	<code>run_coupling_sweep</code> : Validate a finite nonempty one-dimensional D array and force histories. At each D , propagate the chosen comparator and each candidate, reusing it for an identical scheme. Return (S, Q) sample means and initial metrics, (62).
57–80	<code>plot_coupling_sweep</code> : Plot sample means versus physical D and dashed initial values, separately for both metrics. The unsuffixed result is neither final fidelity nor a time-integral quadrature.
83–109	<code>main</code> : Parse current CLI inputs, report relevant estimates before simulation, call the module numerical routine, then plot/export. The main guard preserves importability from notebooks.

3 Comparison with the student implementation and validation

3.1 Exact convention conversions and finite-model changes

Coupling, field phase and state conversion. An exact comparison first needs identical physical modes, bases and dispersion. Call the historical parameter named $\mathbf{g}$ γ_U ; its annihilation coefficient was $\gamma_U \sqrt{|k_m|} e^{-ik_m x_a}$. For sqrt and equal position parameters, matching magnitudes requires

$$D = \sqrt{L} \gamma_U. \quad (73)$$

The box factor means equal numerical old/new couplings need not agree. The factor i is a field phase convention implemented by

$$U = e^{-i\pi \mathcal{N}_f/2}, \quad U a_m U^\dagger = i a_m, \quad U a_m^\dagger U^\dagger = -i a_m^\dagger, \quad H_D = U H_U U^\dagger. \quad (74)$$

Since $U |\mathbf{n}; s\rangle = (-i)^\nu |\mathbf{n}; s\rangle$, applying a ladder before and after the diagonal phase proves the rules. All occupation projectors commute with U , so the equality survives truncation. Matched trajectories also require

$$|\psi_D(0)\rangle = U |\psi_U(0)\rangle, \quad |\psi_D(t)\rangle = U |\psi_U(t)\rangle. \quad (75)$$

Fixed-photon-number preparation changes only by a global phase. Coherent preparation instead requires

$$\alpha_D = -i\alpha_U, \quad (76)$$

including after projection. Keeping the same complex α while changing the Hamiltonian phase is not a matched-state conversion. Correctly related trajectories have equal TLS reductions and number populations; phase-sensitive field observables must be transformed too. Fidelities are invariant when the same U acts on both compared states. Transforming only one member can produce a convention-dependent overlap.

A displaced TLS additionally satisfies

$$H(x) = U_x H(0) U_x^\dagger, \quad U_x = \exp\left(ix \sum_m k_m n_m\right), \quad U_x a_m U_x^\dagger = e^{-ik_m x} a_m. \quad (77)$$

This checks both transition phases simultaneously. It does not alter the Fourier-coordinate interpretation in equation (47).

Packet-width and selection conversion. The historical amplitude was $\exp[-\sigma_U^2(k - k_0)^2/2]$. Equating exponents with equation (15) gives

$$\sigma_k = \frac{1}{\sqrt{2}\sigma_U}, \quad x_0 = x_{\text{photon},U}, \quad k_0 = k_{\text{photon},U}. \quad (78)$$

The old width multiplies momentum differences; the new width is a momentum standard-deviation parameter. Increasing old σ_U narrows the momentum packet. Normalization on identical modes then gives the same coefficients.

Historical selection radii were $w_U \sigma_U$, whereas new radii are $w_D \sigma_k$. Equal numerical radii additionally require

$$w_D = \frac{w_U \sigma_U}{\sigma_k} = \sqrt{2} w_U \sigma_U^2. \quad (79)$$

This is a conversion in the historical numerical units. Envelope conversion alone does not match a selected calculation: the window radii and nearest-mode fallback must also be checked. Current factors mean multiples of the actual momentum width, rather than automatically reproducing old factor values.

Implicit UV regulation and the changed grid. The historical folded grid used L and requested M_U . For even $M_U = 2J$, its retained indices after zero deletion were

$$\{-J+1, \dots, -1, 1, \dots, J\}, \quad |\mathcal{K}_U| = M_U - 1, \quad k_{\max} = J\Delta k = \frac{\pi M_U}{L}. \quad (80)$$

The positive endpoint was unpaired. Full bandwidth is approximately $2\pi M_U/L$ and radial UV approximately $\pi M_U/L$: these are different uses of “cutoff”. Current UV means largest retained $|k|$. At $M_U = 16, L = 20$, old indices were $-7, \dots, -1, 1, \dots, 8$. New explicit $M = 15$ gives $-7, \dots, 0, \dots, 7$; $M = 17$ gives $-8, \dots, 0, \dots, 8$. Neither reproduces the old set. The symmetric cutoff branch also cannot reproduce its unpaired endpoint.

This is an intentional change of finite model, not a stylistic adjustment or a coupling redefinition. The new UV values at $L = 20$ are $7\pi/10$ and $4\pi/5$ respectively. If one wants a fixed physical UV while increasing L and refining momentum spacing, cutoff control states that intention explicitly and lets the count follow. Positive IR excludes zero only by an explicit band choice, not a hard-coded special case.

Parameter migration and compatibility boundaries. The numerical conversion rules can be located directly in the current inputs:

Historical input	Current input	Meaning and required check
<code>length</code>	Atomic <code>L</code>	Same physical length in natural units; the new momentum list must still be checked explicitly.
<code>modes</code>	Config <code>M</code> or cutoffs	Odd actual count under explicit control; it is not the historical pre-deletion counter.
<code>g</code>	Atomic <code>D</code>	$D = \sqrt{L} \gamma_U$ for sqrt and matched units; coherent phase also requires (76).
<code>w_atom, x_atom</code>	Atomic <code>omega_0, x_tls</code>	Same frequency and Hamiltonian phase parameter. Do not inherit the old nonzero default position when intending a TLS at zero.
<code>k_photon, x_photon</code>	Photon <code>k_0, x_0</code>	Same signed packet centre and phase parameter at matched modes.
<code>sigma_photon</code>	Photon <code>sigma_k</code>	Reciprocal-width conversion (78), not a direct rename.
<code>state class</code>	Photon <code>state</code> string	number or coherent ; explicit n or alpha in the photon dictionary.
<code>atom_state</code>	Atomic <code>initial_state</code>	Current preparation interface accepts $g, e, +, -$ labels. The former arbitrary two-component sequence is not an accepted label in this interface.
<code>excitation_cap</code>	Config <code>n_max</code>	Cap meaning depends on the basis: per-mode in full, total in total-cap, ray occupation in truncated.
<code>t, dt</code>	Time <code>T, dt</code>	Same duration and nominal sampling intention, but a changed output-time construction.
<code>hbar, c</code>	Fixed natural units	The current API assumes both equal one; it does not expose arbitrary dimensional constants.

The old special single-mode branch placed a mode at the packet momentum. Current explicit $M = 1$ instead means the zero mode, as specified by the symmetric-grid rule. Legacy classes such as `Config` and the separate Hamiltonian/state subclasses are replaced by the shared configuration and engine. Experiments use the canonical modules in `experiment/`; the historical internal class API and argument syntax are not preserved as aliases. The user’s dictionary-based scattering interface is retained.

Sampling changes also have a concrete consequence. At $T = 4$ and $\text{dt}=0.05$, the old construction requested 80 outputs with spacing $4/79$, whereas the current construction requests 81 outputs with spacing 0.05. A mean over those samples is a different discrete statistic even if both solvers approximate the same continuous trajectory. In the old construction, dt was used to choose a count rather than enforced as an output spacing.

Comparative summary and corrected overlap.

Item	Student implementation	Patch and consequence
Grid	Requested count reduced by zero deletion; unpaired end-point possible.	Exact odd count with zero or an explicit radial band; reports resulting count and effective cutoffs.
Coupling	Historical parameter, real coefficient at zero position.	$iDf_m/\sqrt{L}$ with signed phase. Exact conversion at fixed grid is (73)–(75). Uri notebooks use <code>sqrt</code> .
Packet	Multiplicative width in exponent.	Momentum width in denominator, conversion (78); all selected momentum signs enter preparation.
Assembly	Search among pairs of states.	Dictionary targets and bosonic transitions, $O(Md)$ assembly; shared entries fixed by (33).
Indexing	Separate classes and scheme-specific indices.	Occupation tuples, alternating TLS entries and comparisons of physical configurations.
Cross-basis overlap	Different basis types compared through one-occupied-mode configurations.	All shared occupations in implicit full, physical momentum alignment, and vacuum embedding of absent modes.
Metrics	Global/atomic metrics plus a ratio used as a field proxy.	Only squared state/atomic fidelities; initial differences separately visible.
Times	<code>linspace(0,T,int(T/dt))</code> and output renormalization.	Nominal dt , exact final time, raw norms and explicit tolerances. Sample means may change even with similar trajectories.
Control	Heterogeneous scripts and inputs.	Callable modules, five notebooks and common CLI. Former <code>codex/</code> migrated and removed.

A minimal counterexample to the old cross-type overlap is $|1, 1; g\rangle$ in full and total-cap with $N = 2$. The physical overlap is one; a sum over one occupied mode gives zero. The corrected pair returns both fidelities equal to one. This fixes a comparison, not a propagated trajectory.

At identical grids, number preparations are identical in every basis that contains them, and remain identical for $D = 0$. Coherent projections can have equation (27) below one. Since occupation projectors commute with H_0 , their mutual overlap remains at that initial value during free evolution on the shared grid. This diagnoses preparation differences without mistaking them for interaction-induced error.

Reproducing old figures requires the exact mode list and cap/basis, coupling magnitude and phase, envelope and coherent phase, selection radii, output sampling and statistic. Parameter labels alone cannot establish reproduction when the finite grids or initial projections differ.

3.2 Mathematical verification and limits of the conclusions

Independent finite-dimensional oracles. Tests construct small oscillator/TLS tensor operators independently of the transition loops, select rows/columns for each occupation basis and check

$$H_{\text{code}} = P_{\mathcal{B}} H_{\text{tensor}} P_{\mathcal{B}}, \quad H_{\text{code}} = H_{\text{code}}^{\dagger}. \quad (81)$$

Checks cover three bases, two profiles, RWA on/off and zero/nonzero positions. They also test parity, RWA excitation conservation and the simultaneous annihilation/creation phase identity (77).

An analytic check takes $L = 2\pi$, modes $\{-1, 0, 1\}$, sqrt, $\omega_0 = 1$, RWA, $N \geq 1$ and input $|0; e\rangle$. At zero TLS position,

$$|B\rangle = (|1_-; g\rangle + |1_+; g\rangle)/\sqrt{2}, \quad |\text{dark}\rangle = (|1_-; g\rangle - |1_+; g\rangle)/\sqrt{2}. \quad (82)$$

The dark and zero-mode states decouple. In $(|0; e\rangle, |B\rangle)$,

$$H = I + \begin{pmatrix} 0 & iG \\ -iG & 0 \end{pmatrix}, \quad G = \frac{\sqrt{2}D}{\sqrt{L}}, \quad P_e(t) = \cos^2(Gt). \quad (83)$$

This checks the two-mode factor, box normalization and evolution in all bases. Independent $e^{-iHt}\psi_0$ comparisons check BDF and final-only storage.

Fidelities are tested against explicit small common-space embeddings with reordered grids, selected subsets, caps and shared multimode configurations. Atomic fidelity is checked against equation (58). The (1, 1) case protects the correction; orthogonal field directions with identical TLS protect the distinction between metrics. Photon populations sum to the ket norm, and the total Hamiltonian expectation is checked for conservation. Gaussian, vacuum, coherent and zero-mode preparations are tested separately from evolution.

Integration tests exercise all five experiment module CLIs, their exports, and the callable functions. The five notebooks are checked for valid Python cells and the requested grid/coupling conventions. Tabular diagnostics are checked against the raw arrays, including final-only labels, absent grid equivalences and preservation of norm drift and numerical precision. A documentation rebuild does not replace rerunning the suite after numerical code changes. In an installed development environment, use:

```
1 python -m pytest tests -q
```

Documentation compiles from its sources and regenerates line comments from current files. Derivations added here explain the current model; they do not introduce additional solver features.

Integration error versus projection error. Norm and energy conservation are necessary checks, but two different unitary evolutions can preserve both. Tighter tolerances or an independent exponential reference on small systems test integration error more directly. For an interpolated numerical trajectory with $r(t) = i\dot{v}_{\text{num}} - H_{\mathcal{B}}v_{\text{num}}$, variation of constants gives

$$\|v_{\text{num}}(t) - v(t)\| \leq \|v_{\text{num}}(0) - v(0)\| + \int_0^t \|r(s)\| ds. \quad (84)$$

The engine reports sampled diagnostics, not an evaluated residual integral; this equation specifies what a residual-based certificate would require.

For projection, take a larger finite space with Hamiltonian H , smaller projector P , and $Q = I - P$. Let ψ solve the larger evolution and ψ_P solve PHP . For $e = P\psi - \psi_P$,

$$i\dot{e} = PHPe + PHQ\psi, \quad \|e(t)\| \leq \|e(0)\| + \int_0^t \|PHQ\psi(s)\| ds. \quad (85)$$

The driving term depends on omitted dynamics, not just final boundary population. Normalized coherent projections can give $e(0) \neq 0$. Cap comparisons are useful, but a small final boundary population alone is not a rigorous dynamical error certificate. The modules do not evaluate this bound.

Regulator convergence and finite-box scattering. The checks establish finite-model consistency, not physical continuum convergence. A study still needs separate variations of

$$N \rightarrow N', \quad k_{\text{UV}} \rightarrow k'_{\text{UV}}, \quad k_{\text{IR}} \rightarrow k'_{\text{IR}}, \quad L \rightarrow L', \quad (\text{rtol}, \text{atol}) \rightarrow (\text{rtol}', \text{atol}'). \quad (86)$$

Truncated omits multimode configurations at every N ; its large-cap limit is not full Fock space. Conversely, RWA number preparations can occupy a small invariant excitation sector in which larger caps make no difference. That agreement does not test arbitrary high-photon dynamics.

Changing L at fixed explicit count changes spacing, UV, resonant-mode availability and $D/\sqrt{L}$ together. Holding physical D and cutoffs fixed while changing L is a different discretization study. For a finite smooth band the associated Riemann sum is

$$\frac{1}{L} \sum_{k_m \in \mathcal{K}} f_m^2 \rightarrow \frac{1}{2\pi} \int_{k_{\text{IR}} \leq |k| \leq k_{\text{UV}}} f(k)^2 dk. \quad (87)$$

This explains why per-mode $L^{-1/2}$ normalization and a growing mode count belong together. It does not prove convergence as UV increases without bound.

Free phases $e^{-i|j|\Delta kt}$ recur after $t = L$. Interactions can change exact recurrence times, but a periodic packet can return to the interaction region after circulating around the box. Scattering observations should separate incoming/outgoing packets before such returns and be checked in larger boxes. Increasing T without this check can turn single-passage scattering into repeated interaction. Initial negative/zero components and non-RWA multiphoton sectors must also be accounted for before assigning asymptotic coefficients.

Historical plots become numerical references only after their actual grids, preparations, coupling conversions and statistics have been reconstructed. The equations, visible notebook inputs and configuration metadata make these comparisons reproducible without treating a changed finite model as a code-style refinement.